

ΕΡΓΑΣΙΑ 2

Γραμμική & Συνδυαστική Βελτιστοποίηση

Διορθωμένη και συμπληρωμένη έκδοση

Εμμανουήλ Βήχος

AM: 1100830

Τμήμα Ηλεκτρολόγων Μηχανικών & Τεχνολογίας Υπολογιστών

Περιεχόμενα

Σημείωμα για την παρούσα έκδοση	2
1 Ανάλυση ευαισθησίας από το βέλτιστο tableau	3
1.1 Περιγραφή της βέλτιστης λύσης	3
1.2 (α) Αλλαγή του c_1 σε 1	4
1.3 (β) Αλλαγή του c^T σε $[1, 2, 3, 4]$	5
1.4 (γ) Αλλαγή του b_3 σε 11	6
1.5 (δ) Αλλαγή του b^T σε $[3, -2, 4]$	7
1.6 Συγκεντρωτικός πίνακας	7
2 Μοντελοποίηση προβλήματος ανάθεσης εργασιών	8
2.1 Μοντελοποίηση	8
2.2 Επίλυση	8
2.3 Ερμηνεία και δυϊκές τιμές	9
3 Δυϊκό πρόβλημα και συμπληρωματική χαλαρότητα	10
3.1 (α) Κατασκευή και επίλυση του δυϊκού	10
3.2 (β) Η συμπληρωματική λύση του πρωτεύοντος	11
4 Επίλυση με τον αλγόριθμο Dual Simplex	13
4.1 Γιατί ταιριάζει ο dual simplex	13
4.2 Εκτέλεση	13
4.3 Εκφυλισμός, μοναδικότητα και δυϊκές λύσεις	14
5 Επίλυση με τον αλγόριθμο Branch & Bound	15
5.1 Τα τρία κριτήρια αποκοπής	15
5.2 Εκτέλεση	15
6 Μοντέλο μικτού ακέραιου προγραμματισμού	17
6.1 Γραμμικοποίηση	17
6.2 Επίλυση	18
6.3 Ανεξάρτητη επαλήθευση με ανάλυση περιπτώσεων	18
7 Πρόβλημα σακιδίου: B&B, δυϊκό και συμπληρωματική χαλαρότητα	19
7.1 (α) Best-first Branch & Bound	19
7.2 (β) Δυϊκό του χαλαρωμένου και συμπληρωματική χαλαρότητα	20
7.3 (γ) Ακέραιο δυϊκό και έλεγχος της CS	21
A' Παράρτημα: πηγαίος κώδικας	23
A'.1 ex_1.py — ανάλυση ευαισθησίας με πλήρη tableaux	23
A'.2 ex_2.py — ανάθεση εργασιών, δυϊκές τιμές, σύγκριση με ακέραιο	25
A'.3 ex_3.py — δυϊκό και συμπληρωματική χαλαρότητα	26
A'.4 ex_4.py — dual simplex με tableaux	28
A'.5 ex_5.py — Branch & Bound με ρητά κριτήρια αποκοπής	30
A'.6 ex_6.py — MIP με big-M και αναλυτική επαλήθευση	32
A'.7 ex_7.py — σακίδιο: best-first B&B, δυϊκό, CS	34

Σημείωμα για την παρούσα έκδοση

Αναθεωρημένη έκδοση της Εργασίας 2. Ξαναδουλεύοντας τις ασκήσεις εντόπισα αρκετά σημεία στα οποία η αρχική μου υποβολή ήταν ελλιπής ή λανθασμένη — σε αρκετά από αυτά δινόταν μόνο το *αποτέλεσμα*, χωρίς τη διαδικασία που ζητούσε ρητά η εκφώνηση. Πιστεύω πως έλειπαν τα παρακάτω:

Άσκηση	Τι προστέθηκε / διορθώθηκε
1	Προστίθενται όλες οι επαναλήψεις της Simplex σε μορφή tableau για τα ερωτήματα (α), (β), (γ) και (δ) — η εκφώνηση ζητούσε «πρόσθετες επαναλήψεις της Simplex» και στην αρχική υποβολή δινόταν μόνο η τελική λύση. Διορθώνεται και η διατύπωση στο (β): μετά τη διαταραχή παραβιάζει τη βελτιστότητα μόνο η x_2 (όχι «αρκετές» μεταβλητές). Προστίθενται οι σκιώδεις τιμές $y = c_B B^{-1}$.
2	Έλειπαν οι δυϊκές τιμές με οικονομική ερμηνεία, η επαλήθευση της ισχυρής δυϊκότητας και η σύγκριση με το αντίστοιχο <i>ακέραιο</i> πρόβλημα, που ποσοτικοποιεί το όφελος της κλασματικής εκτέλεσης.
3	Αποδεικνύεται ότι η βέλτιστη λύση του δυϊκού είναι μοναδική και ότι η δυϊκή κορυφή είναι εκφυλισμένη (4 ενεργά υπερεπίπεδα στον $\mathbb{R}^3$). Περιγράφεται όλο το σύνολο των βέλτιστων λύσεων του πρωτεύοντος (ακμή) και οι δύο κορυφές του, όχι μία μόνο λύση.
4	Έλεπε η πλήρης παρουσίαση των tableaux κάθε επανάληψης, η τεκμηρίωση της μοναδικότητας της βέλτιστης λύσης, και σύνδεση του εκφυλισμού της πρωτεύουσας κορυφής με την <i>πολλαπλότητα</i> των βέλτιστων δυϊκών λύσεων.
5	Διορθώνεται το δέντρο B&B : στην αρχική υποβολή ο κλάδος $x_1 = 0$ αναπτύχθηκε πλήρως, ενώ με σωστή εφαρμογή του κριτηρίου φράγματος αποκόπτεται αμέσως. Το δέντρο περιορίζεται από 5 σε 3 κόμβους.
6	Δεν είχε τεκμηριωθεί η επιλογή του M (δεν αρκεί «αρκετά μεγάλο»): αποδεικνύεται ότι κάθε βέλτιστη λύση έχει $x_1 \leq 11.25$, $x_2 \leq 9$ και επαληθεύεται αριθμητικά ότι το αποτέλεσμα δεν εξαρτάται από το M . Προστίθεται πλήρης αναλυτική επίλυση κατά περιπτώσεις, ανεξάρτητη από τον solver.
7	Διορθώνεται ο χαρακτηρισμός του κόμβου με λύση $(0, 1, 0, 0, 1, 0, 1)$: είναι <i>ακέραια επικτή λύση</i> ($z = 214$) που γίνεται προσωρινό incumbent, όχι «pruned». Προστίθεται ρητά η σειρά επιλογής των κόμβων κατά best-first, όπως ζητούσε η εκφώνηση.
Γενικά	Διορθώνονται τα σφάλματα στοιχειοθεσίας της αρχικής υποβολής (λατινικές λέξεις που είχαν μεταγραφεί σε ελληνικά, π.χ. «Στρογγ Δυναμιψ», «σολνερ», «νς», «μαξ»). Ο κώδικας παρατίθεται ως κείμενο αντί για στιγμιότυπα οθόνης. Όλα τα tableaux υπολογίζονται σε ακριβή ρητή αριθμητική.

1 Ανάλυση ευαισθησίας από το βέλτιστο tableau

Το πρόβλημα

$$\max z = 2x_1 + x_2 + 4x_3 - x_4$$

$$(\Pi_1) \ x_1 + 2x_2 + x_3 - 3x_4 \leq 8, \quad (\Pi_2) \ -x_2 + x_3 + 2x_4 \leq 0, \quad (\Pi_3) \ 2x_1 + 7x_2 - 5x_3 - 10x_4 \leq 21, \quad x \geq 0.$$

Με μεταβλητές χαλάρωσης $x_5, x_6, x_7 \geq 0$ το σύστημα γίνεται

$$x_1 + 2x_2 + x_3 - 3x_4 + x_5 = 8, \quad -x_2 + x_3 + 2x_4 + x_6 = 0, \quad 2x_1 + 7x_2 - 5x_3 - 10x_4 + x_7 = 21.$$

Δοσμένο βέλτιστο tableau								
Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	$\frac{1}{2}$	$\frac{5}{2}$	0	1	$\frac{3}{2}$	0	8
x_4	0	$-\frac{1}{2}$	$\frac{1}{2}$	1	0	$\frac{1}{2}$	0	0
x_7	0	1	-5	0	-2	2	1	5
$c_j - z_j$	0	$-\frac{1}{2}$	$-\frac{1}{2}$	0	-2	$-\frac{5}{2}$	0	$z = 16$

1.1 Περιγραφή της βέλτιστης λύσης

Βασικές / μη βασικές μεταβλητές. Βασικές: x_1, x_4, x_7 . Μη βασικές (ίσες με μηδέν): x_2, x_3, x_5, x_6 .

$$x^* = (x_1, x_2, x_3, x_4) = (8, 0, 0, 0), \quad x_5 = 0, \ x_6 = 0, \ x_7 = 5, \quad z^* = 2(8) + 0 + 0 - 0 = \mathbf{16}.$$

Όλα τα $c_j - z_j \leq 0$, που για πρόβλημα μεγιστοποίησης επιβεβαιώνει τη βελτιστότητα.

Ο βέλτιστος βασικός πίνακας. Οι στήλες των x_1, x_4, x_7 στον αρχικό $[A \mid I]$ δίνουν

$$B = \begin{pmatrix} 1 & -3 & 0 \\ 0 & 2 & 0 \\ 2 & -10 & 1 \end{pmatrix}, \quad B^{-1} = \begin{pmatrix} 1 & \frac{3}{2} & 0 \\ 0 & \frac{1}{2} & 0 \\ -2 & 2 & 1 \end{pmatrix}.$$

Ο B^{-1} διαβάζεται απευθείας από τις στήλες των αρχικών μεταβλητών χαλάρωσης x_5, x_6, x_7 του βέλτιστου tableau. Επαλήθευση:

$$B^{-1}b = \begin{pmatrix} 1 & \frac{3}{2} & 0 \\ 0 & \frac{1}{2} & 0 \\ -2 & 2 & 1 \end{pmatrix} \begin{pmatrix} 8 \\ 0 \\ 21 \end{pmatrix} = \begin{pmatrix} 8 \\ 0 \\ 5 \end{pmatrix} = (x_1, x_4, x_7)^\top \checkmark$$

Οι σκιάσεις τιμές είναι

$$y^\top = c_B^\top B^{-1} = (2, -1, 0) B^{-1} = (2, \frac{5}{2}, 0),$$

δηλαδή $y_1 = 2, y_2 = \frac{5}{2}, y_3 = 0$ — σε συμφωνία με τη γραμμή $c_j - z_j$ των x_5, x_6, x_7 (που είναι $-y_1, -y_2, -y_3$).

Δεσμευτικοί και μη δεσμευτικοί περιορισμοί.

$(\Pi_1) : 8 + 0 + 0 - 0 = 8 = 8$	$\Rightarrow x_5 = 0$	δεσμευτικός
$(\Pi_2) : -0 + 0 + 0 = 0 = 0$	$\Rightarrow x_6 = 0$	δεσμευτικός
$(\Pi_3) : 2(8) + 0 - 0 - 0 = 16 < 21$	$\Rightarrow x_7 = 5$	μη δεσμευτικός

Γεωμετρική περιγραφή της βέλτιστης κορυφής. Στον $\mathbb{R}^4$ μια μη εκφυλισμένη κορυφή ορίζεται από την τομή ακριβώς 4 γραμμικά ανεξάρτητων υπερεπιπέδων. Στο σημείο $(8, 0, 0, 0)$ είναι ενεργά:

$$\underbrace{\Pi_1, \Pi_2}_{2 \text{ περιορισμοί}}, \quad \underbrace{x_2 = 0, x_3 = 0, x_4 = 0}_{3 \text{ υπερεπίπεδα μη αρνητικότητας}} \implies \mathbf{5 \text{ ενεργά υπερεπίπεδα.}}$$

Αφού τέμνονται $5 > 4$ υπερεπίπεδα, η βέλτιστη κορυφή είναι **εκφυλισμένη**. Αλγεβρικά αυτό φαίνεται από το ότι η *βασική* μεταβλητή x_4 έχει τιμή 0. Γεωμετρικά, η κορυφή βρίσκεται πάνω στην ημιευθεία του άξονα x_1 (σημείο $x_1 = 8$) και είναι υπερπροσδιορισμένη: οποιαδήποτε 4 από τα 5 υπερεπίπεδα αρκούν για να την ορίσουν. Συνέπεια του εκφυλισμού είναι ότι το *δυσκό* πρόβλημα έχει πολλαπλές βέλτιστες λύσεις, και ότι μια διαταραχή του b μπορεί να καταστήσει τη λύση μη εφικτή ακόμη και για πολύ μικρή μεταβολή.

1.2 (α) Αλλαγή του c_1 σε 1

Η x_1 είναι *βασική*, άρα η μεταβολή του c_1 αλλάζει το c_B και επομένως ολόκληρη τη γραμμή $c_j - z_j$. Η εφικτότητα $(B^{-1}b)$ δεν θίγεται. Με $c_B = (c_1, c_4, c_7) = (1, -1, 0)$:

$$y^\top = c_B^\top B^{-1} = (1, 1, 0),$$

	x_2	x_3	x_5	x_6
$z_j - c_j = y^\top A_j - c_j$	$(2 - 1) - 1 = 0$	$(1 + 1) - 4 = \mathbf{-2}$	$1 - 0 = 1$	$1 - 0 = 1$

Το $z_3 - c_3 = -2 < 0$, δηλαδή $c_3 - z_3 = +2 > 0$: **χάνεται η βελτιστότητα** και εισέρχεται η x_3 . Εκτελούμε primal simplex.

Επανάληψη 0.

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	$\frac{1}{2}$	$\frac{5}{2}$	0	1	$\frac{3}{2}$	0	8
x_4	0	$-\frac{1}{2}$	$\frac{1}{2}$	1	0	$\frac{1}{2}$	0	0
x_7	0	1	-5	0	-2	2	1	5
$c_j - z_j$	0	0	2	0	-1	-1	0	$z = 8$

Ratio test: $\frac{8}{5/2} = \frac{16}{5}$ και $\frac{0}{1/2} = \mathbf{0}$. Εξέρχεται η x_4 με λόγο 0: πρόκειται για **εκφυλισμένο pivot** — άμεση συνέπεια του εκφυλισμού της κορυφής. Το σημείο δεν μετακινείται και το z παραμένει 8.

Επανάληψη 1.

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	3	0	-5	1	-1	0	8
x_3	0	-1	1	2	0	1	0	0
x_7	0	-4	0	10	-2	7	1	5
$c_j - z_j$	0	2	0	-4	-1	-3	0	$z = 8$

Εισέρχεται η x_2 : μόνη θετική στήλη είναι η γραμμή x_1 με $\frac{8}{3}$, άρα εξέρχεται η x_1 .

Επανάληψη 2 (τελική).

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_2	$\frac{1}{3}$	1	0	$-\frac{5}{3}$	$\frac{1}{3}$	$-\frac{1}{3}$	0	$\frac{8}{3}$
x_3	$\frac{1}{3}$	0	1	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{2}{3}$	0	$\frac{8}{3}$
x_7	$\frac{4}{3}$	0	0	$\frac{10}{3}$	$-\frac{2}{3}$	$\frac{17}{3}$	1	$\frac{47}{3}$
$c_j - z_j$	$-\frac{2}{3}$	0	0	$-\frac{2}{3}$	$-\frac{5}{3}$	$-\frac{7}{3}$	0	$z = \frac{40}{3}$

$$x^* = \left(0, \frac{8}{3}, \frac{8}{3}, 0\right), \quad z^* = 1(0) + 1\frac{8}{3} + 4\frac{8}{3} - 0 = \frac{40}{3} \approx 13.33.$$

Έλεγχος: $\Pi_1 : 0 + \frac{16}{3} + \frac{8}{3} = 8 \checkmark$ (δεσμ.), $\Pi_2 : -\frac{8}{3} + \frac{8}{3} + 0 = 0 \checkmark$ (δεσμ.), $\Pi_3 : \frac{56}{3} - \frac{40}{3} = \frac{16}{3} < 21 \checkmark$.

1.3 (β) Αλλαγή του $c^\top$ σε $[1, 2, 3, 4]$

Νέο $c_B = (c_1, c_4, c_7) = (1, 4, 0)$, άρα $y^\top = c_B^\top B^{-1} = (1, \frac{7}{2}, 0)$ και

	x_2	x_3	x_5	x_6
$c_j - z_j$	$2 - (2 - \frac{7}{2}) = +\frac{7}{2}$	$3 - (1 + \frac{7}{2}) = -\frac{3}{2}$	$0 - 1 = -1$	$0 - \frac{7}{2} = -\frac{7}{2}$

Διόρθωση σε σχέση με την αρχική υποβολή. Εκεί αναφερόταν ότι «αρκετά από τα μειωμένα κόστη παραβιάζουν τη βελτιστότητα». Στην πραγματικότητα παραβιάζει μόνο ένα: η στήλη x_2 , με $c_2 - z_2 = +\frac{7}{2} > 0$. Όλα τα υπόλοιπα παραμένουν ≤ 0 . Η λύση παραμένει εφικτή (το b δεν άλλαξε), άρα συνεχίζουμε με primal simplex.

Επανάληψη 0. Εισέρχεται η x_2 . Ratio test: $\frac{8}{1/2} = 16$ (γραμμή x_1), $\frac{5}{1} = 5$ (γραμμή x_7)· η γραμμή x_4 έχει αρνητικό συντελεστή. Εξέρχεται η x_7 .

Επανάληψη 1.

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	0	5	0	2	$\frac{1}{2}$	$-\frac{1}{2}$	$\frac{11}{2}$
x_4	0	0	-2	1	-1	$\frac{3}{2}$	$\frac{1}{2}$	$\frac{5}{2}$
x_2	0	1	-5	0	-2	2	1	5
$c_j - z_j$	0	0	16	0	6	$-\frac{21}{2}$	$-\frac{7}{2}$	$z = \frac{51}{2}$

Εισέρχεται η x_3 (μεγαλύτερο $c_j - z_j = 16$)· μόνη θετική στήλη η γραμμή x_1 με λόγο $\frac{11/2}{5} = \frac{11}{10}$. Εξέρχεται η x_1 .

Επανάληψη 2 (τελική).

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_3	$\frac{1}{5}$	0	1	0	$\frac{2}{5}$	$\frac{1}{10}$	$-\frac{1}{10}$	$\frac{11}{10}$
x_4	$\frac{2}{5}$	0	0	1	$-\frac{1}{5}$	$\frac{17}{10}$	$\frac{3}{10}$	$\frac{47}{10}$
x_2	1	1	0	0	0	$\frac{5}{2}$	$\frac{1}{2}$	$\frac{21}{2}$
$c_j - z_j$	$-\frac{16}{5}$	0	0	0	$-\frac{2}{5}$	$-\frac{121}{10}$	$-\frac{19}{10}$	$z = \frac{431}{10}$

$$x^* = \left(0, \frac{21}{2}, \frac{11}{10}, \frac{47}{10}\right) = (0, 10.5, 1.1, 4.7), \quad z^* = \frac{431}{10} = 43.1.$$

Και οι τρεις περιορισμοί είναι τώρα δεσμευτικοί: $\Pi_1 : 21 + 1.1 - 14.1 = 8 \checkmark$, $\Pi_2 : -10.5 + 1.1 + 9.4 = 0 \checkmark$, $\Pi_3 : 73.5 - 5.5 - 47 = 21 \checkmark$. Μαζί με το $x_1 = 0$ έχουμε ακριβώς 4 ενεργά υπερεπίπεδα, άρα η νέα βέλτιστη κορυφή δεν είναι εκφυλισμένη.

1.4 (γ) Αλλαγή του b_3 σε 11

Η μεταβολή του δεξιού μέλους δεν επηρεάζει τη γραμμή $c_j - z_j$ (άρα η λύση παραμένει δυϊκά εφικτή / βέλτιστη ως προς τα κόστη), αλλά αλλάζει τις τιμές των βασικών μεταβλητών:

$$B^{-1}b_{\text{νέο}} = \begin{pmatrix} 1 & \frac{3}{2} & 0 \\ 0 & \frac{1}{2} & 0 \\ -2 & 2 & 1 \end{pmatrix} \begin{pmatrix} 8 \\ 0 \\ 11 \end{pmatrix} = \begin{pmatrix} 8 \\ 0 \\ -5 \end{pmatrix}.$$

Το $x_7 = -5 < 0$: **χάνεται η εφικτότητα**. Εφαρμόζουμε *dual simplex*.

Επανάληψη 0.

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	$\frac{1}{2}$	$\frac{5}{2}$	0	1	$\frac{3}{2}$	0	8
x_4	0	$-\frac{1}{2}$	$\frac{1}{2}$	1	0	$\frac{1}{2}$	0	0
x_7	0	1	-5	0	-2	2	1	-5
$c_j - z_j$	0	$-\frac{1}{2}$	$-\frac{1}{2}$	0	-2	$-\frac{5}{2}$	0	$z = 16$

Εξέρχεται η x_7 (μοναδικό αρνητικό RHS). Ratio test μόνο στις στήλες με *αρνητικό* στοιχείο στη γραμμή αυτή:

$$x_3 : \left| \frac{-1/2}{-5} \right| = \mathbf{0.1}, \quad x_5 : \left| \frac{-2}{-2} \right| = 1 \implies \text{εισέρχεται η } x_3.$$

Επανάληψη 1.

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	1	0	0	0	$\frac{5}{2}$	$\frac{1}{2}$	$\frac{11}{2}$
x_4	0	$-\frac{2}{5}$	0	1	$-\frac{1}{5}$	$\frac{7}{10}$	$\frac{1}{10}$	$-\frac{1}{2}$
x_3	0	$-\frac{1}{5}$	1	0	$\frac{2}{5}$	$-\frac{2}{5}$	$-\frac{1}{5}$	1
$c_j - z_j$	0	$-\frac{3}{5}$	0	0	$-\frac{9}{5}$	$-\frac{27}{10}$	$-\frac{1}{10}$	$z = \frac{31}{2}$

Εξέρχεται η x_4 . Ratio test: $x_2 : \left| \frac{-3/5}{-2/5} \right| = \mathbf{1.5}$, $x_5 : \left| \frac{-9/5}{-1/5} \right| = 9 \implies \text{εισέρχεται η } x_2$.

Επανάληψη 2 (τελική).

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	0	0	$\frac{5}{2}$	$-\frac{1}{2}$	$\frac{17}{4}$	$\frac{3}{4}$	$\frac{17}{4}$
x_2	0	1	0	$-\frac{5}{2}$	$\frac{1}{2}$	$-\frac{7}{4}$	$-\frac{1}{4}$	$\frac{5}{4}$
x_3	0	0	1	$-\frac{1}{2}$	$\frac{1}{2}$	$-\frac{3}{4}$	$-\frac{1}{4}$	$\frac{5}{4}$
$c_j - z_j$	0	0	0	$-\frac{3}{2}$	$-\frac{3}{2}$	$-\frac{15}{4}$	$-\frac{1}{4}$	$z = \frac{59}{4}$

Όλα τα RHS ≥ 0 και όλα τα $c_j - z_j \leq 0$:

$$x^* = (4.25, 1.25, 1.25, 0), \quad z^* = \frac{59}{4} = 14.75.$$

Έλεγχος: $\Pi_1 : 4.25 + 2.5 + 1.25 = 8 \checkmark$, $\Pi_2 : -1.25 + 1.25 + 0 = 0 \checkmark$, $\Pi_3 : 8.5 + 8.75 - 6.25 = 11 \checkmark$ (τώρα δεσμευτικός).

1.5 (δ) Αλλαγή του $b^\top$ σε $[3, -2, 4]$

$$B^{-1}b_{\text{νέο}} = \begin{pmatrix} 1 & \frac{3}{2} & 0 \\ 0 & \frac{1}{2} & 0 \\ -2 & 2 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ -2 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \\ -6 \end{pmatrix}.$$

Δύο βασικές μεταβλητές γίνονται αρνητικές ($x_4 = -1$, $x_7 = -6$). Dual simplex:

Επανάληψη 0. Εξέρχεται η x_7 (RHS = -6, το πιο αρνητικό). Ratio test: $x_3 : 0.1$ (ελάχιστος), $x_5 : 1 \Rightarrow$ εισέρχεται η x_3 .

Επανάληψη 1.

Βάση	x_1	x_2	x_3	x_4	x_5	x_6	x_7	RHS
x_1	1	1	0	0	0	$\frac{5}{2}$	$\frac{1}{2}$	-3
x_4	0	$-\frac{2}{5}$	0	1	$-\frac{1}{5}$	$\frac{7}{10}$	$\frac{1}{10}$	$-\frac{8}{5}$
x_3	0	$-\frac{1}{5}$	1	0	$\frac{2}{5}$	$-\frac{2}{5}$	$-\frac{1}{5}$	$\frac{6}{5}$
$c_j - z_j$	0	$-\frac{3}{5}$	0	0	$-\frac{9}{5}$	$-\frac{27}{10}$	$-\frac{1}{10}$	$z = \frac{2}{5}$

Η γραμμή της x_1 έχει RHS = -3 < 0, αλλά όλα τα στοιχεία της στις μη βασικές στήλες είναι ≥ 0 ($x_2 : 1$, $x_6 : \frac{5}{2}$, $x_7 : \frac{1}{2}$). Δεν υπάρχει επιτρεπτό ρινο: αυτό είναι ακριβώς το κριτήριο τερματισμού του dual simplex που πιστοποιεί **ΜΗ ΕΦΙΚΤΟΤΗΤΑ** του πρωτεύοντος.

Ανεξάρτητη αλγεβρική απόδειξη. Η γραμμή αυτή αντιστοιχεί στην ανισότητα $x_1 + x_2 + \frac{5}{2}x_6 + \frac{1}{2}x_7 = -3$ με $x \geq 0$, που είναι προφανώς αδύνατη. Ισοδύναμα, από τον Π_2 έχουμε $x_2 \geq x_3 + 2x_4 + 2 \geq 2$. Θέτοντας $x_1 = x_3 = 0$ (κάθε θετική τιμή τους χειροτερεύει την κατάσταση), ο Π_1 δίνει $2x_2 - 3x_4 \leq 3$, ενώ $x_2 \geq 2x_4 + 2$ συνεπάγεται $2x_2 - 3x_4 \geq 4x_4 + 4 - 3x_4 = x_4 + 4 \geq 4 > 3$ — άτοπο. Άρα δεν υπάρχει εφικτή λύση.

1.6 Συγκεντρωτικός πίνακας

Ερώτ.	Διαταραχή	Τι χάνεται	Μέθοδος	Νέα λύση
—	—	—	—	$x = (8, 0, 0, 0)$, $z = 16$
(α)	$c_1 = 1$	βελτιστότητα	primal simplex (2 επαν.)	$(0, \frac{8}{3}, \frac{8}{3}, 0)$, $z = \frac{40}{3}$
(β)	$c = [1, 2, 3, 4]$	βελτιστότητα	primal simplex (2 επαν.)	$(0, 10.5, 1.1, 4.7)$, $z = 43.1$
(γ)	$b_3 = 11$	εφικτότητα	dual simplex (2 επαν.)	$(4.25, 1.25, 1.25, 0)$, $z = 14.75$
(δ)	$b = [3, -2, 4]$	εφικτότητα	dual simplex	μη εφικτό

2 Μοντελοποίηση προβλήματος ανάθεσης εργασιών

Δεδομένα

Εργαστήριο	A	B	Γ	Δ	Κόστος/ώρα k_i	Διαθέσιμος χρόνος
1	32	151	72	118	89	160
2	39	147	61	126	81	160
3	46	155	57	121	84	160

2.1 Μοντελοποίηση

Μεταβλητές απόφασης.

x_{ij} = το κλάσμα της εργασίας j που εκτελείται στο εργαστήριο i , $i \in \{1, 2, 3\}$, $j \in \{A, B, \Gamma, \Delta\}$.

Είναι *συνεχείς* στο $[0, 1]$, αφού η εκφώνηση επιτρέπει ρητά την κλασματική εκτέλεση· έχουμε $3 \times 4 = 12$ μεταβλητές.

Αντικειμενική συνάρτηση. Αν η εργασία j απαιτεί t_{ij} ώρες στο εργαστήριο i και το εργαστήριο χρεώνει k_i ανά ώρα, η εκτέλεση ολόκληρης της εργασίας κοστίζει $c_{ij} = t_{ij}k_i$, άρα η εκτέλεση κλάσματος x_{ij} κοστίζει $c_{ij}x_{ij}$:

$$\min Z = \sum_{i=1}^3 \sum_{j \in \{A, B, \Gamma, \Delta\}} c_{ij} x_{ij}, \quad c_{ij} = t_{ij} k_i.$$

Πίνακας κόστους c_{ij} (€ για ολόκληρη την εργασία)

Εργαστήριο	A	B	Γ	Δ
1	2848	13439	6408	10502
2	3159	11907	4941	10206
3	3864	13020	4788	10164

Περιορισμοί.

$$\sum_{i=1}^3 x_{ij} = 1, \quad \forall j \quad (\text{κάθε εργασία εκτελείται πλήρως}) \quad (1)$$

$$\sum_j t_{ij} x_{ij} \leq 160, \quad \forall i \quad (\text{διαθέσιμος χρόνος εργαστηρίου}) \quad (2)$$

$$0 \leq x_{ij} \leq 1, \quad \forall i, j \quad (3)$$

Σημείωση: το άνω φράγμα $x_{ij} \leq 1$ συνεπάγεται ήδη από τον (1) μαζί με τη μη αρνητικότητα, οπότε είναι πλεονάζον· το κρατάμε για σαφήνεια.

2.2 Επίλυση

Το πρόβλημα λύθηκε με τη βιβλιοθήκη PuLP και τον επιλυτή CBC.

Εργαστήριο	A	B	Γ	Δ	Ώρες που χρησιμοποιούνται
1	1	0	0	0.045586	37.379/160
2	0	1	0	0.103175	160/160
3	0	0	1	0.851240	160/160

$$Z^* = 29,726.74 \text{ €}.$$

Εργασία A $\rightarrow$ 100% στο Εργαστήριο 1· B $\rightarrow$ 100% στο 2· Γ $\rightarrow$ 100% στο 3· Δ $\rightarrow$ 85.12% στο 3, 10.32% στο 2, 4.56% στο 1.

2.3 Ερμηνεία και δυϊκές τιμές

Αν δεν υπήρχε περιορισμός χρόνου, κάθε εργασία θα πήγαινε ολόκληρη στο φθηνότερο εργαστήριο: A $\rightarrow$ 1, B $\rightarrow$ 2, Γ $\rightarrow$ 3, Δ $\rightarrow$ 3. Όμως οι Γ και Δ μαζί απαιτούν $57 + 121 = 178 > 160$ ώρες στο Εργαστήριο 3. Ο επιλυτής κρατά την Γ ολόκληρη εκεί (γι' αυτήν το 3 είναι σαφώς φθηνότερο) και «σπάει» την Δ, της οποίας το κόστος διαφέρει ελάχιστα μεταξύ των τριών εργαστηρίων.

Περιορισμός	Δυϊκή τιμή	Ερμηνεία
χρόνος Εργ. 1	0	μη δεσμευτικός (μένουν 122.6 αχρησιμοποίητες ώρες)
χρόνος Εργ. 2	-2.3492	μία επιπλέον ώρα στο Εργ. 2 μειώνει το κόστος κατά 2.35 €
χρόνος Εργ. 3	-2.7934	μία επιπλέον ώρα στο Εργ. 3 μειώνει το κόστος κατά 2.79 €
εκτέλεση A	2848.00	οριακό κόστος της εργασίας A
εκτέλεση B	12252.33	οριακό κόστος της εργασίας B
εκτέλεση Γ	4947.22	οριακό κόστος της εργασίας Γ
εκτέλεση Δ	10502.00	οριακό κόστος της εργασίας Δ

Η τιμή της δυϊκής αντικειμενικής είναι $\sum_j u_j + 160 \sum_i w_i = 29,726.74 = Z^*$, δηλαδή επαληθεύεται η ισχυρή δυϊκότητα.

Πόσο αξίζει η κλασματική εκτέλεση; Αν απαγορευόταν το «σπάσιμο» των εργασιών (δηλαδή $x_{ij} \in \{0, 1\}$), η βέλτιστη ανάθεση θα ήταν A $\rightarrow$ 1, B $\rightarrow$ 2, Γ $\rightarrow$ 3, Δ $\rightarrow$ 1 με κόστος 30,045 €. Άρα η δυνατότητα κλασματικής εκτέλεσης εξοικονομεί 318.26 € (περίπου 1.1%), και ταυτόχρονα επιτρέπει να μείνει το πρόβλημα ΓΠ αντί για ακέραιο.

3 Δυϊκό πρόβλημα και συμπληρωματική χαλαρότητα

Το πρωτεύον

$$\min z = 32x_2 + 50x_3 - 19x_5$$

$$(\Pi_1) 15(x_1 + x_2 + x_3) + x_5 = 40, \quad (\Pi_2) 12x_1 - 90x_2 + 14x_4 \geq 18, \quad (\Pi_3) x_4 \leq 11, \quad x_j \geq 0.$$

3.1 (α) Κατασκευή και επίλυση του δυϊκού

Για πρόβλημα ελαχιστοποίησης με μη αρνητικές μεταβλητές, οι κανόνες αντιστοιχίας δίνουν:

Πρωτεύων περιορισμός	Τύπος	Δυϊκή μεταβλητή
Π_1	ισότητα	y_1 ελεύθερη
Π_2	$\geq$ σε min	$y_2 \geq 0$
Π_3	$\leq$ σε min	$y_3 \leq 0$

Επειδή όλες οι $x_j \geq 0$, κάθε δυϊκός περιορισμός (ένας ανά στήλη j) είναι τύπου $\leq c_j$:

$$\max W = 40y_1 + 18y_2 + 11y_3$$

$$(x_1) : 15y_1 + 12y_2 \leq 0$$

$$(x_2) : 15y_1 - 90y_2 \leq 32$$

$$(x_3) : 15y_1 \leq 50$$

$$(x_4) : 14y_2 + y_3 \leq 0$$

$$(x_5) : y_1 \leq -19$$

$$y_1 \in \mathbb{R}, y_2 \geq 0, y_3 \leq 0.$$

Η επίλυση με PuLP/CBC δίνει:

$$y^* = (-19, 0, 0), \quad W^* = 40(-19) + 18(0) + 11(0) = -760.$$

Απόδειξη μοναδικότητας (δεν αρκεί ο solver). Από τον περιορισμό (x_5) έχουμε $y_1 \leq -19$, άρα $40y_1 \leq -760$. Από τον (x_1) , $12y_2 \leq -15y_1$, και από τον (x_4) , $y_3 \leq -14y_2$. Επομένως

$$W = 40y_1 + 18y_2 + 11y_3 \leq 40y_1 + 18y_2 - 154y_2 = 40y_1 - 136y_2 \leq -760,$$

όπου η τελευταία ανισότητα ισχύει επειδή $y_2 \geq 0$. Η ισότητα απαιτεί ταυτόχρονα $y_1 = -19$, $y_2 = 0$ και $y_3 = 0$. Άρα η βέλτιστη δυϊκή λύση είναι **μοναδική**.

Δεσμευτικοί δυϊκοί περιορισμοί στο $y^* = (-19, 0, 0)$.

Περιορισμός	LHS	RHS	Περίσσεια	Κατάσταση
$(x_1) 15y_1 + 12y_2 \leq 0$	-285	0	285	μη δεσμευτικός
$(x_2) 15y_1 - 90y_2 \leq 32$	-285	32	317	μη δεσμευτικός
$(x_3) 15y_1 \leq 50$	-285	50	335	μη δεσμευτικός
$(x_4) 14y_2 + y_3 \leq 0$	0	0	0	δεσμευτικός
$(x_5) y_1 \leq -19$	-19	-19	0	δεσμευτικός

Γεωμετρική περιγραφή της δυϊκής κορυφής. Στον $\mathbb{R}^3$ μια κορυφή απαιτεί 3 γραμμικά ανεξάρτητα ενεργά υπερεπίπεδα. Στο y^* είναι ενεργά **τέσσερα**:

$$y_1 = -19 \text{ (περιορ. } x_5), \quad 14y_2 + y_3 = 0 \text{ (περιορ. } x_4), \quad y_2 = 0, \quad y_3 = 0.$$

Η δυϊκή βέλτιστη κορυφή είναι εκφυλισμένη (τέσσερα ενεργά υπερεπίπεδα αντί για τρία): τα $y_2 = 0$ και $y_3 = 0$ συνεπάγονται ήδη τον περιορισμό $14y_2 + y_3 = 0$. Αυτό δεν είναι λεπτομέρεια — ο εκφυλισμός του δυϊκού είναι ακριβώς η αιτία για την οποία το *πρωτεύον* έχει, όπως θα δούμε, άπειρες βέλτιστες λύσεις.

3.2 (β) Η συμπληρωματική λύση του πρωτεύοντος

Οι σχέσεις συμπληρωματικής χαλαρότητας. Για ένα ζεύγος εφικτών λύσεων (x, y) , η βελτιστότητα και των δύο είναι ισοδύναμη με:

$$(CS1) \quad x_j(c_j - A_j^\top y) = 0 \quad \forall j \quad \text{δηλ. } x_j > 0 \Rightarrow \text{ο δυϊκός } j \text{ δεσμευτικός} \quad (4)$$

$$(CS2) \quad y_i(A_i x - b_i) = 0 \quad \forall i \quad \text{δηλ. } y_i \neq 0 \Rightarrow \text{ο πρωτεύων } i \text{ δεσμευτικός} \quad (5)$$

Ισοδύναμα (αντιθετοαντιστροφή της CS1): αν ο δυϊκός περιορισμός j είναι *μη* δεσμευτικός, τότε αναγκαστικά $x_j = 0$.

Βήμα 1 — από τους μη δεσμευτικούς δυϊκούς. Οι περιορισμοί που αντιστοιχούν στις x_1, x_2, x_3 έχουν γνήσια θετική περίσσεια, άρα

$$x_1^* = x_2^* = x_3^* = 0.$$

Βήμα 2 — από τον Π_1 . Αντικαθιστώντας στον Π_1 : $15(0 + 0 + 0) + x_5 = 40 \Rightarrow x_5^* = 40$.

Βήμα 3 — τι λέει (και τι δεν λέει) η CS2. Είναι $y_1 = -19 \neq 0$, άρα ο Π_1 πρέπει να είναι δεσμευτικός — ισχύει αυτομάτως, αφού είναι ισότητα. Οι $y_2 = 0$ και $y_3 = 0$ δεν δίνουν καμία πληροφορία για τους Π_2, Π_3 : η CS2 ικανοποιείται ανεξαρτήτως του αν είναι δεσμευτικοί.

Βήμα 4 — η x_4 . Η x_4 δεν εμφανίζεται στην αντικειμενική ($c_4 = 0$), άρα η τιμή της δεν επηρεάζει το z . Περιορίζεται μόνο από την εφικτότητα:

$$(\Pi_2) : 14x_4 \geq 18 \Rightarrow x_4 \geq \frac{9}{7} \approx 1.2857, \quad (\Pi_3) : x_4 \leq 11.$$

$$z^* = 32(0) + 50(0) - 19(40) = -760 = W^* \quad (\text{ισχυρή δυϊκότητα } \checkmark)$$

Το σύνολο των βέλτιστων λύσεων του πρωτεύοντος είναι η ακμή

$$X^* = \left\{ (0, 0, 0, x_4, 40) : \frac{9}{7} \leq x_4 \leq 11 \right\},$$

με δύο κορυφές:

$$x^{(1)} = \left(0, 0, 0, \frac{9}{7}, 40 \right) \text{ (δεσμευτικός ο } \Pi_2), \quad x^{(2)} = (0, 0, 0, 11, 40) \text{ (δεσμευτικός ο } \Pi_3).$$

Η ύπαρξη πολλαπλών βέλτιστων πρωτευουσών λύσεων *προβλέπεται* από τον εκφυλισμό της δυϊκής κορυφής που εντοπίσαμε στο ερώτημα (α) — και αντίστροφα, η μοναδικότητα της δυϊκής λύσης εγγυάται ότι *κάθε* βέλτιστη πρωτεύουσα λύση έχει $x_1 = x_2 = x_3 = 0$, δηλαδή ότι το X^* που βρήκαμε είναι πλήρες.

Γεωμετρική περιγραφή. Για την κορυφή $x^{(2)} = (0, 0, 0, 11, 40)$: στον $\mathbb{R}^5$ χρειάζονται 5 ενεργά υπερεπίπεδα και έχουμε ακριβώς

$$\Pi_1 \text{ (ισότητα), } \Pi_3 (x_4 = 11), \quad x_1 = 0, \quad x_2 = 0, \quad x_3 = 0,$$

ενώ ο Π_2 είναι μη δεσμευτικός ($14(11) = 154 > 18$) — σε πλήρη συμφωνία με $y_2^* = 0$.

4 Επίλυση με τον αλγόριθμο Dual Simplex

Το πρόβλημα

$$\min z = 3x_1 + 4x_2 + 6x_3 + 7x_4 + x_5$$

$$(\Pi_1) \ 2x_1 - x_2 + x_3 + 6x_4 - 5x_5 \geq 6, \quad (\Pi_2) \ x_1 + x_2 + 2x_3 + x_4 + 2x_5 \geq 3, \quad x \geq 0.$$

4.1 Γιατί ταιριάζει ο dual simplex

Το πρόβλημα είναι ελαχιστοποίησης με όλα τα $c_j \geq 0$ και όλους τους περιορισμούς $\geq$. Πολλαπλασιάζοντας με -1 και προσθέτοντας $s_1, s_2 \geq 0$:

$$-2x_1 + x_2 - x_3 - 6x_4 + 5x_5 + s_1 = -6, \quad -x_1 - x_2 - 2x_3 - x_4 - 2x_5 + s_2 = -3.$$

Η αρχική βάση $\{s_1, s_2\}$ δίνει $s_1 = -6, s_2 = -3$: *πρωτεύοντως μη εφικτή* αλλά *δυϊκά εφικτή* (όλα τα $c_j - z_j = c_j \geq 0$). Αυτή είναι ακριβώς η ιδανική εκκίνηση για τον dual simplex, ο οποίος διατηρεί τη δυϊκή εφικτότητα και αποκαθιστά σταδιακά την πρωτεύουσα.

4.2 Εκτέλεση

Επανάληψη 0.

Βάση	x_1	x_2	x_3	x_4	x_5	s_1	s_2	RHS
s_1	-2	1	-1	-6	5	1	0	-6
s_2	-1	-1	-2	-1	-2	0	1	-3
$c_j - z_j$	3	4	6	7	1	0	0	$z = 0$

Εξερχόμενη: s_1 (πιο αρνητικό RHS = -6). **Ratio test** $\left| \frac{c_j - z_j}{a_{rj}} \right|$ μόνο για $a_{rj} < 0$:

$$x_1 : \frac{3}{-2} = 1.5, \quad x_3 : \frac{6}{-1} = 6, \quad x_4 : \frac{7}{-6} \approx 1.167 \text{ (ελάχιστος)}.$$

Εισερχόμενη: x_4 · στοιχείο οδηγός -6.

Επανάληψη 1.

Βάση	x_1	x_2	x_3	x_4	x_5	s_1	s_2	RHS
x_4	$\frac{1}{3}$	$-\frac{1}{6}$	$\frac{1}{6}$	1	$-\frac{5}{6}$	$-\frac{1}{6}$	0	1
s_2	$-\frac{2}{3}$	$-\frac{7}{6}$	$-\frac{11}{6}$	0	$-\frac{17}{6}$	$-\frac{1}{6}$	1	-2
$c_j - z_j$	$\frac{2}{3}$	$\frac{31}{6}$	$\frac{29}{6}$	0	$\frac{41}{6}$	$\frac{7}{6}$	0	$z = 7$

Εξερχόμενη: s_2 (RHS = -2). **Ratio test:**

$$x_1 : \left| \frac{2/3}{-2/3} \right| = 1, \quad x_2 : \left| \frac{31/6}{-7/6} \right| = \frac{31}{7} \approx 4.43, \quad x_3 : \frac{29}{11} \approx 2.64, \quad x_5 : \frac{41}{17} \approx 2.41, \quad s_1 : 7.$$

Εισερχόμενη: x_1 · στοιχείο οδηγός $-\frac{2}{3}$.

Επανάληψη 2 (τελική).

Βάση	x_1	x_2	x_3	x_4	x_5	s_1	s_2	RHS
x_4	0	$-\frac{3}{4}$	$-\frac{3}{4}$	1	$-\frac{9}{4}$	$-\frac{1}{4}$	$\frac{1}{2}$	0
x_1	1	$\frac{7}{4}$	$\frac{11}{4}$	0	$\frac{17}{4}$	$\frac{1}{4}$	$-\frac{3}{2}$	3
$c_j - z_j$	0	4	3	0	4	1	1	$z = 9$

Όλα τα RHS ≥ 0 (πρωτεύουσα εφικτότητα) και όλα τα $c_j - z_j \geq 0$ (δυϊκή εφικτότητα): ο αλγόριθμος τερματίζει.

$$x^* = (3, 0, 0, 0, 0), \quad z^* = 3(3) = \mathbf{9}.$$

Έλεγχος: $\Pi_1 : 2(3) = 6 \geq 6 \checkmark$ (δεσμευτικός), $\Pi_2 : 3 \geq 3 \checkmark$ (δεσμευτικός).

4.3 Εκφυλισμός, μοναδικότητα και δυϊκές λύσεις

Εκφυλισμός. Η βασική μεταβλητή x_4 έχει τιμή 0 στο βέλτιστο tableau. Γεωμετρικά, στον $\mathbb{R}^5$ μια κορυφή χρειάζεται 5 ενεργά υπερεπίπεδα, ενώ στο $x^* = (3, 0, 0, 0, 0)$ είναι ενεργά

$$x_2 = 0, x_3 = 0, x_4 = 0, x_5 = 0, \Pi_1, \Pi_2 \implies \mathbf{6} > 5.$$

Άρα η βέλτιστη κορυφή είναι **εκφυλισμένη**.

Μοναδικότητα της πρωτεύουσας λύσης. Τα $c_j - z_j$ όλων των μη βασικών μεταβλητών είναι γνησίως θετικά ($x_2 : 4, x_3 : 3, x_5 : 4, s_1 : 1, s_2 : 1$). Άρα κάθε μετακίνηση αυξάνει το κόστος: η βέλτιστη λύση είναι **μοναδική**.

Το δυϊκό έχει πολλαπλές βέλτιστες λύσεις. Το δυϊκό είναι

$$\max W = 6u_1 + 3u_2 \quad \text{με} \quad \begin{aligned} 2u_1 + u_2 &\leq 3, & -u_1 + u_2 &\leq 4, & u_1 + 2u_2 &\leq 6, \\ 6u_1 + u_2 &\leq 7, & -5u_1 + 2u_2 &\leq 1, & u_1, u_2 &\geq 0. \end{aligned}$$

Από τη CS, αφού $x_1 > 0$, ο πρώτος δυϊκός περιορισμός είναι δεσμευτικός: $2u_1 + u_2 = 3$. Αντικαθιστώντας, $W = 6u_1 + 3(3 - 2u_1) = 9$ *ανεξάρτητα* από το u_1 . Οι υπόλοιποι περιορισμοί δίνουν $6u_1 - 9u_1 + 6 \leq \dots$ και τελικά $u_1 \in [\frac{5}{9}, 1]$:

u_1	$\frac{5}{9}$	$\frac{3}{4}$	1
$u_2 = 3 - 2u_1$	$\frac{17}{9}$	$\frac{3}{2}$	1
W	9	9	9

Αυτό είναι ακριβώς η αναμενόμενη συνέπεια του *εκφυλισμού* της πρωτεύουσας βέλτιστης κορυφής: εκφυλισμένη πρωτεύουσα κορυφή $\Leftrightarrow$ μη μοναδικές δυϊκές τιμές.

5 Επίλυση με τον αλγόριθμο Branch & Bound

Το πρόβλημα

$$\min z = 12x_1 + 45x_2 + 67x_3 + x_4$$

$$(\Pi_1) \ 4x_1 + 2x_2 - x_4 \leq 10, \quad (\Pi_2) \ 6x_1 + 19x_3 \geq 5, \quad x_2, x_3, x_4 \geq 0, \quad x_1 \in \{0, 1\}, \quad x_3 \in \mathbb{Z}_{\geq 0}.$$

Πρόκειται για *μικτό* ακέραιο πρόβλημα: οι x_1 (δυαδική) και x_3 (ακέραια) υπόκεινται σε ακεραιότητα, ενώ οι x_2, x_4 είναι συνεχείς.

5.1 Τα τρία κριτήρια αποκοπής

Σε κάθε κόμβο λύνεται μόνο το χαλαρωμένο ΓΠ. Ο κλάδος αποκόπτεται (fathomed) όταν:

1. το χαλαρωμένο ΓΠ είναι **μη εφικτό**·
2. η λύση του είναι **ακέραια** (τότε ενημερώνεται το *incumbent*)·
3. $z_{LP} \geq z_{\text{incumbent}}$ — **αποκοπή από φράγμα** (πρόβλημα ελαχιστοποίησης), αφού κανένας απόγονος δεν μπορεί να δώσει καλύτερη λύση από το φράγμα του κόμβου.

Η επιλογή του επόμενου κόμβου γίνεται με *best-first*: διακλαδίζεται πάντα ο ζωντανός κόμβος με το *μικρότερο* z_{LP} .

5.2 Εκτέλεση

Κόμβος 1 (ρίζα). Χαλάρωση: $x_1 \in [0, 1], x_3 \geq 0$ συνεχείς. Ο Π_2 απαιτεί «μονάδες» από τις x_1 ή x_3 : το οριακό κόστος είναι $12/6 = 2$ ανά μονάδα μέσω x_1 και $67/19 \approx 3.53$ μέσω x_3 , οπότε ο επιλυτής χρησιμοποιεί μόνο την x_1 :

$$x = (\frac{5}{6}, 0, 0, 0) = (0.8333, 0, 0, 0), \quad z_{LP} = 10.$$

Η x_1 δεν είναι ακέραια $\Rightarrow$ διακλάδωση στο x_1 .

Κόμβος 3 ($x_1 = 1$). $\Pi_2 : 6 + 19x_3 \geq 5$ ικανοποιείται με $x_3 = 0$. Άρα

$$x = (1, 0, 0, 0), \quad z_{LP} = 12.$$

Όλες οι μεταβλητές ακέραιες $\Rightarrow$ **incumbent** $z^{\text{best}} = 12$.

Κόμβος 2 ($x_1 = 0$). $\Pi_2 : 19x_3 \geq 5 \Rightarrow x_3 \geq \frac{5}{19}$, άρα

$$x = (0, 0, \frac{5}{19}, 0) = (0, 0, 0.2632, 0), \quad z_{LP} = \frac{335}{19} \approx 17.63.$$

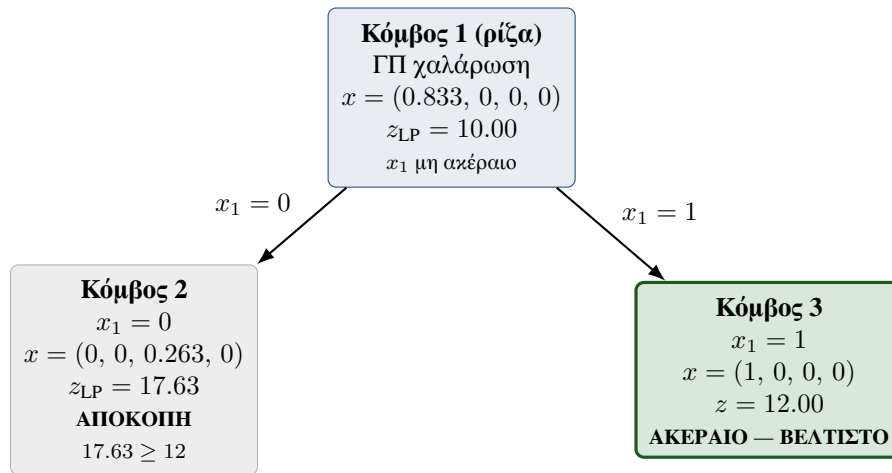
Διόρθωση σε σχέση με την αρχική υποβολή. Εκεί ο κόμβος αυτός αναπτύχθηκε περαιτέρω, με διακλάδωση στο x_3 σε δύο ακόμη κόμβους. Αυτό είναι περιττό: αφού $z_{LP} = 17.63 \geq 12 = z^{\text{best}}$, ο κλάδος **αποκόπτεται από το φράγμα**. Κανένας ακέραιος απόγονός του δεν μπορεί να δώσει $z < 17.63$, πολύ περισσότερο $z < 12$. Η αποκοπή αυτή είναι η ουσία του B&B — χωρίς αυτήν ο αλγόριθμος εκφυλίζεται σε πλήρη απαρίθμηση.

Δεν απομένουν ζωντανοί κόμβοι: ο αλγόριθμος τερματίζει με **3 κόμβους** (αντί για 5 της αρχικής υποβολής).

$$x^* = (x_1, x_2, x_3, x_4) = (1, 0, 0, 0), \quad z^* = 12.$$

Έλεγχος: $\Pi_1 : 4(1) + 0 - 0 = 4 \leq 10 \checkmark$, $\Pi_2 : 6(1) + 0 = 6 \geq 5 \checkmark$, $x_1 = 1 \in \{0, 1\} \checkmark$, $x_3 = 0 \in \mathbb{Z}_{\geq 0} \checkmark$.

Κενό ακεραιότητας: $z^* - z_{LP}^{\text{ρίζα}} = 12 - 10 = 2$.



Σχήμα 1: Δέντρο Branch & Bound. Μπλε: ρίζα. Γκρι: κόμβος που αποκόπτεται από το φράγμα. Πράσινο: η βέλτιστη ακέραια λύση.

Σχόλιο. Οι μεταβλητές x_2 και x_4 μηδενίζονται σε κάθε κόμβο: έχουν θετικούς συντελεστές κόστους (45 και 1) και δεν βοηθούν σε κανέναν περιορισμό (η x_2 σφίγγει τον Π_1 , η x_4 τον χαλαρώνει αλλά ο Π_1 δεν είναι ποτέ δεσμευτικός). Είναι χρήσιμο να το παρατηρήσουμε, γιατί εξηγεί γιατί το δέντρο είναι τόσο μικρό.

6 Μοντέλο μικτού ακέραιου προγραμματισμού

Το πρόβλημα

$$\min f_1(x_1) + f_2(x_2), \quad f_1(x_1) = \begin{cases} 5 + 4x_1, & x_1 > 0 \\ 0, & x_1 = 0 \end{cases} \quad f_2(x_2) = \begin{cases} 3 + 5x_2, & x_2 > 0 \\ 0, & x_2 = 0 \end{cases}$$

1. $x_1 \geq 5 \vee x_2 \geq 5$;
2. ισχύει τουλάχιστον ένας από τους $2x_1 + x_2 \geq 12$, $x_1 + x_2 \geq 10$, $x_1 + 2x_2 \geq 12$;
3. $|x_1 - x_2| \in \{0, 5, 10\}$;
4. $x_1, x_2 \geq 0$.

Υπάρχουν τρεις πηγές μη γραμμικότητας: *σταθερά κόστη* (ασυνεχείς συναρτήσεις κόστους), *διαζευκτικοί περιορισμοί* (H) και ένα *διακριτό σύνολο* τιμών για την απόλυτη διαφορά.

6.1 Γραμμικοποίηση

(α) **Σταθερά κόστη.** Εισάγουμε $y_1, y_2 \in \{0, 1\}$ με $y_i = 1$ όταν $x_i > 0$:

$$\min Z = 5y_1 + 4x_1 + 3y_2 + 5x_2, \quad x_1 \leq My_1, \quad x_2 \leq My_2.$$

Ο περιορισμός $x_i \leq My_i$ επιβάλλει $y_i = 1$ όποτε $x_i > 0$. Το αντίστροφο ($x_i = 0 \Rightarrow y_i = 0$) δεν χρειάζεται να επιβληθεί: αφού ελαχιστοποιούμε και ο συντελεστής του y_i είναι θετικός, ο επιλυτής θα θέσει $y_i = 0$ όποτε του επιτρέπεται.

(β) **Διαζευκτικός περιορισμός 1.** Με $z_1, z_2 \in \{0, 1\}$:

$$z_1 + z_2 \geq 1, \quad x_1 \geq 5z_1, \quad x_2 \geq 5z_2.$$

Αν $z_1 = 1$ τότε $x_1 \geq 5$ · αν $z_2 = 1$ τότε $x_2 \geq 5$ · και τουλάχιστον ένα από τα δύο ισχύει.

(γ) **«Τουλάχιστον ένας από τρεις».** Με $w_1, w_2, w_3 \in \{0, 1\}$ και big-M:

$$\begin{aligned} 2x_1 + x_2 &\geq 12 - M(1 - w_1) \\ w_1 + w_2 + w_3 &\geq 1, \quad x_1 + x_2 \geq 10 - M(1 - w_2) \\ x_1 + 2x_2 &\geq 12 - M(1 - w_3) \end{aligned}$$

Όταν $w_r = 1$ ο αντίστοιχος περιορισμός ενεργοποιείται· όταν $w_r = 0$ ο όρος $-M$ τον καθιστά τετριμμένα ικανοποιημένο.

(δ) $|x_1 - x_2| \in \{0, 5, 10\}$. Με $a_0, a_5, a_{10} \in \{0, 1\}$, $a_0 + a_5 + a_{10} = 1$ και $v = 5a_5 + 10a_{10}$, και με δυαδική s για το πρόσημο:

$$x_1 - x_2 \leq v + M(1 - s), \quad x_1 - x_2 \geq v - M(1 - s), \quad x_2 - x_1 \leq v + Ms, \quad x_2 - x_1 \geq v - Ms.$$

Αν $s = 1$ οι δύο πρώτες δίνουν $x_1 - x_2 = v$ (και οι άλλες δύο χαλαρώνουν)· αν $s = 0$ οι δύο τελευταίες δίνουν $x_2 - x_1 = v$. Σε κάθε περίπτωση $|x_1 - x_2| = v \in \{0, 5, 10\}$.

Τεκμηρίωση της τιμής του M . Δεν αρκεί «ένα μεγάλο M »: αν το M είναι πολύ μικρό αποκόπτονται εφικτές λύσεις, ενώ αν είναι υπερβολικά μεγάλο η χαλάρωση ΓΠ γίνεται αδύναμη και το B&B αργεί. Εδώ:

- Άνω φράγμα για τα x_i : η λύση $x = (10, 0)$ είναι εφικτή με κόστος $5 + 40 = 45$. Άρα σε κάθε βέλτιστη λύση ισχύει $4x_1 \leq Z \leq 45$ και $5x_2 \leq 45$, δηλαδή $x_1 \leq 11.25$ και $x_2 \leq 9$. Επομένως το φράγμα $x_1, x_2 \leq 20$ δεν αποκόπτει καμία βέλτιστη λύση.
- Με $x_i \leq 20$, η ποσότητα $|x_1 - x_2| - v$ δεν ξεπερνά τα 20 σε απόλυτη τιμή, άρα κάθε $M \geq 25$ είναι ασφαλές. Επιλέγουμε $M = 40$.

Ο κώδικας επαληθεύει ότι το αποτέλεσμα είναι ίδιο για $M \in \{25, 40, 100, 1000\}$, δηλαδή η διατύπωση δεν εξαρτάται από την επιλογή.

6.2 Επίλυση

$$x_1^* = \frac{17}{3} \approx 5.667, \quad x_2^* = \frac{2}{3} \approx 0.667, \quad Z^* = \mathbf{34}.$$

Ενεργές δυαδικές: $y_1 = y_2 = 1, z_1 = 1, w_1 = 1, a_5 = 1, s = 1$.

Έλεγχος: $f_1(\frac{17}{3}) = 5 + \frac{68}{3} = \frac{83}{3}, f_2(\frac{2}{3}) = 3 + \frac{10}{3} = \frac{19}{3}$, άθροισμα = $\frac{102}{3} = 34$.

Έλεγχος των αρχικών (μη γραμμικών) περιορισμών.

- 1) $x_1 = \frac{17}{3} \approx 5.67 \geq 5 \checkmark$
- 2) $2x_1 + x_2 = \frac{34}{3} + \frac{2}{3} = 12 \geq 12 \checkmark$ (δεσμευτικός); $x_1 + x_2 = \frac{19}{3} \approx 6.33$; $x_1 + 2x_2 = 7$
- 3) $|x_1 - x_2| = \left| \frac{17}{3} - \frac{2}{3} \right| = 5 \in \{0, 5, 10\} \checkmark$
- 4) $x_1, x_2 \geq 0 \checkmark$

6.3 Ανεξάρτητη επαλήθευση με ανάλυση περιπτώσεων

Επειδή το πρόβλημα έχει μόνο δύο συνεχείς μεταβλητές, μπορεί να λυθεί και αναλυτικά. Ο περιορισμός 3 καθηγώνει τη διαφορά $x_1 - x_2$ σε μία από έξι τιμές $(0, \pm 5, \pm 10)$, οπότε μένει ένα μονοδιάστατο γραμμικό πρόβλημα που λύνεται ακριβώς.

Περίπτωση	Βέλτιστο σημείο	Ποιος περιορισμός δένει	Z
$x_2 = 0, x_1 = 5$	—	μη εφικτό (κανένας της ομάδας 2)	—
$x_2 = 0, x_1 = 10$	$(10, 0)$	$2x_1 \geq 12$	$5 + 40 = 45$
$x_1 = 0, x_2 = 5$	—	μη εφικτό	—
$x_1 = 0, x_2 = 10$	$(0, 10)$	$x_1 + 2x_2 \geq 12$	$3 + 50 = 53$
$ x_1 - x_2 = 0$	$(5, 5)$	$x_1 \geq 5$ (περιορ. 1)	$8 + 20 + 25 = 53$
$x_1 - x_2 = 5$	$(\frac{17}{3}, \frac{2}{3})$	$2x_1 + x_2 \geq 12$	34
$x_2 - x_1 = 5$	$(\frac{2}{3}, \frac{17}{3})$	$x_1 + 2x_2 \geq 12$	39
$x_1 - x_2 = 10$	$\rightarrow (10, 0)$	—	45
$x_2 - x_1 = 10$	$\rightarrow (0, 10)$	—	53

Γιατί κερδίζει η περίπτωση $x_1 - x_2 = 5$. Γράφοντας $x_1 = x_2 + 5$, το κόστος γίνεται $Z = 8 + 4(x_2 + 5) + 5x_2 = 28 + 9x_2$, δηλαδή αύξουσα συνάρτηση του x_2 . Ο μόνος περιορισμός που εμποδίζει το x_2 να πέσει στο 0 είναι ο πιο «χαλαρός» της ομάδας 2, ο $2x_1 + x_2 \geq 12$, που δίνει $3x_2 + 10 \geq 12 \Rightarrow x_2 \geq \frac{2}{3}$. Άρα $Z = 28 + 9 \cdot \frac{2}{3} = 34$. Η συμμετρική περίπτωση $x_2 - x_1 = 5$ δίνει $Z = 33 + 9x_1$ με $x_1 \geq \frac{2}{3}$, δηλαδή 39 — χειρότερη, γιατί η x_2 έχει ακριβότερο μεταβλητό κόστος (5 έναντι 4) και εδώ είναι η μεγάλη μεταβλητή.

Η αναλυτική ανάλυση επιβεβαιώνει πλήρως τον επιλυτή: $Z^* = 34$ στο $x^* = (\frac{17}{3}, \frac{2}{3})$.

7 Πρόβλημα σακιδίου: B&B, δυϊκό και συμπληρωματική χαλαρότητα

Το πρόβλημα

$$\max z = 12x_1 + 12x_2 + 9x_3 + 15x_4 + 90x_5 + 26x_6 + 112x_7$$

$$3x_1 + 4x_2 + 3x_3 + 3x_4 + 15x_5 + 13x_6 + 16x_7 \leq 35, \quad x_i \in \{0, 1\}.$$

i	1	2	3	4	5	6	7
p_i	12	12	9	15	90	26	112
w_i	3	4	3	3	15	13	16
p_i/w_i	4.00	3.00	3.00	5.00	6.00	2.00	7.00

Σειρά κατά φθίνοντα λόγο: $x_7 \succ x_5 \succ x_4 \succ x_1 \succ x_2 \sim x_3 \succ x_6$.

7.1 (α) Best-first Branch & Bound

Το φράγμα. Σε κάθε κόμβο λύνεται το *κλασματικό* knapsack ($0 \leq x_i \leq 1$), το οποίο λύνεται βέλτιστα με τον άπληστο κανόνα: γεμίζουμε κατά φθίνοντα λόγο p_i/w_i και σπάμε μόνο το τελευταίο αντικείμενο. Η τιμή του είναι έγκυρο άνω φράγμα για κάθε ακέραιο απόγονο.

Στρατηγική. Best-first (jumptracking): από όλους τους ζωντανούς κόμβους διακλαδίζεται αυτός με το *μεγαλύτερο* z_{LP} . Ένας κόμβος αποκόπτεται όταν $z_{LP} \leq z^{best}$.

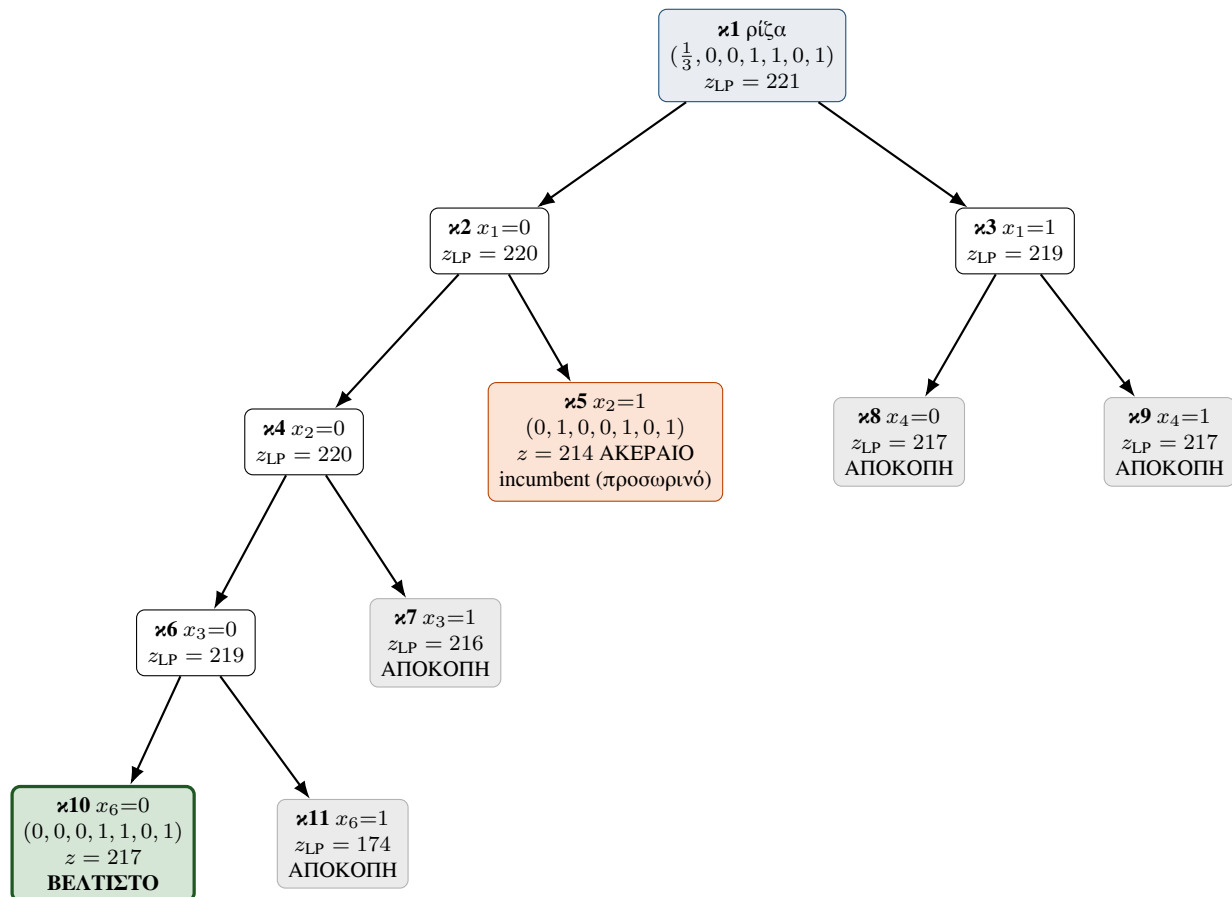
κ	γονέας	δέσμευση	λύση χαλαρωμένου ΓΠ	z_{LP}	ενέργεια
1	—	ρίζα	$(\frac{1}{3}, 0, 0, 1, 1, 0, 1)$	221	διακλάδωση στο x_1
2	1	$x_1 = 0$	$(0, \frac{1}{4}, 0, 1, 1, 0, 1)$	220	διακλάδωση στο x_2
3	1	$x_1 = 1$	$(1, 0, 0, \frac{1}{3}, 1, 0, 1)$	219	διακλάδωση στο x_4
4	2	$x_2 = 0$	$(0, 0, \frac{1}{3}, 1, 1, 0, 1)$	220	διακλάδωση στο x_3
5	2	$x_2 = 1$	$(0, 1, 0, 0, 1, 0, 1)$	214	ΑΚΕΡΑΙΟ $\rightarrow$ incumbent = 214
6	4	$x_3 = 0$	$(0, 0, 0, 1, 1, \frac{1}{13}, 1)$	219	διακλάδωση στο x_6
7	4	$x_3 = 1$	$(0, 0, 1, \frac{1}{3}, 1, 0, 1)$	216	αποκοπή ($216 \leq 217$)
8	3	$x_4 = 0$	$(1, \frac{1}{4}, 0, 0, 1, 0, 1)$	217	αποκοπή ($217 \leq 217$)
9	3	$x_4 = 1$	$(1, 0, 0, 1, \frac{13}{15}, 0, 1)$	217	αποκοπή ($217 \leq 217$)
10	6	$x_6 = 0$	$(0, 0, 0, 1, 1, 0, 1)$	217	ΑΚΕΡΑΙΟ $\rightarrow$ incumbent = 217
11	6	$x_6 = 1$	$(0, 0, 0, 0, \frac{2}{5}, 1, 1)$	174	αποκοπή ($174 \leq 217$)

Σειρά επιλογής κόμβων (best-first) — ζητούμενο της εκφώνησης.

βήμα	κόμβος που επιλέγεται	ενέργεια
1	$\kappa 1$ ($z_{LP} = 221$)	διακλάδωση στο $x_1 \rightarrow \kappa 2, \kappa 3$
2	$\kappa 2$ (220)	διακλάδωση στο $x_2 \rightarrow \kappa 4, \kappa 5$ (incumbent = 214)
3	$\kappa 4$ (220)	διακλάδωση στο $x_3 \rightarrow \kappa 6, \kappa 7$
4	$\kappa 3$ (219)	διακλάδωση στο $x_4 \rightarrow \kappa 8, \kappa 9$
5	$\kappa 6$ (219)	διακλάδωση στο $x_6 \rightarrow \kappa 10$ (incumbent = 217), $\kappa 11$
6	$\kappa 8$ (217)	αποκοπή από φράγμα
7	$\kappa 9$ (217)	αποκοπή από φράγμα
8	$\kappa 7$ (216)	αποκοπή από φράγμα

Παρατηρήστε ότι οι κόμβοι εξετάζονται με φθίνουσα σειρά φράγματος 221, 220, 220, 219, 219, 217, 217, 216 — αυτό ακριβώς σημαίνει «best first».

Διόρθωση σε σχέση με την αρχική υποβολή. Ο κόμβος 5 ($x_1 = 0, x_2 = 1$) είχε χαρακτηριστεί εκεί ως «pruned». Στην πραγματικότητα η λύση του $(0, 1, 0, 0, 1, 0, 1)$ είναι **ακέραια και εφικτή** με $z = 214$: είναι ο πρώτος υποψήφιος και γίνεται προσωρινό incumbent, το οποίο αργότερα αντικαθίσταται από το 217 του κόμβου 10. Ένας κόμβος αποκόπτεται από φράγμα μόνο όταν η λύση του παραμένει κλασματική και το φράγμα δεν υπερβαίνει το incumbent — κάτι που ισχύει για τους κόμβους 7, 8, 9, 11.



Σχήμα 2: Δέντρο Branch & Bound με στρατηγική best-first. Μπλε: ρίζα. Πορτοκαλί: ακέραια λύση που γίνεται προσωρινό incumbent. Γκρι: αποκοπή από φράγμα. Πράσινο: η βέλτιστη ακέραια λύση.

$$x^* = (0, 0, 0, 1, 1, 0, 1), \quad \text{αντικείμενα } \{4, 5, 7\}, \quad \text{βάρος} = 3 + 15 + 16 = 34 \leq 35,$$

$$z^* = 15 + 90 + 112 = \mathbf{217}.$$

Άνω φράγμα ρίζας $z_{LP} = 221$, άρα κενό ακεραιότητας = 4.

7.2 (β) Δυϊκό του χαλαρωμένου και συμπληρωματική χαλαρότητα

Πρωτεύον (χαλαρωμένο).

$$\max \sum_{i=1}^7 p_i x_i \quad \text{με} \quad \sum_{i=1}^7 w_i x_i \leq 35 \quad (\rightarrow u), \quad x_i \leq 1 \quad (\rightarrow v_i), \quad x_i \geq 0.$$

Δυϊκό.

$$\min W = 35u + \sum_{i=1}^7 v_i \quad \text{με} \quad w_i u + v_i \geq p_i \quad (i = 1, \dots, 7), \quad u \geq 0, v_i \geq 0.$$

Σχέσεις συμπληρωματικής χαλαρότητας.

$$u > 0 \implies \sum_i w_i x_i = 35 \quad (\text{γεμάτο σακίδιο}) \quad (6)$$

$$v_i > 0 \implies x_i = 1 \quad (\text{δεσμευτικό άνω όριο}) \quad (7)$$

$$x_i > 0 \implies w_i u + v_i = p_i \quad (\text{δεσμευτικός δυϊκός}) \quad (8)$$

$$x_i < 1 \implies v_i = 0 \quad (\text{μη δεσμευτικό άνω όριο}) \quad (9)$$

Παραγωγή της δυϊκής λύσης. Από τη λύση του χαλαρωμένου $x^{\text{LP}} = (\frac{1}{3}, 0, 0, 1, 1, 0, 1)$:

- $x_1 = \frac{1}{3} \in (0, 1)$: από την (17) είναι $v_1 = 0$ και από την (16) $w_1 u + v_1 = p_1$, άρα $3u = 12 \Rightarrow u^* = 4$.
- $x_4 = 1$: $3(4) + v_4 = 15 \Rightarrow v_4 = 3$.
- $x_5 = 1$: $15(4) + v_5 = 90 \Rightarrow v_5 = 30$.
- $x_7 = 1$: $16(4) + v_7 = 112 \Rightarrow v_7 = 48$.
- $x_2 = x_3 = x_6 = 0$ και $x_i < 1$, άρα $v_2 = v_3 = v_6 = 0$. επαληθεύουμε ότι οι δυϊκοί περιορισμοί ικανοποιούνται: $4(4) = 16 \geq 12$, $3(4) = 12 \geq 9$, $13(4) = 52 \geq 26$ ✓.

$$u^* = 4, \quad v^* = (0, 0, 0, 3, 30, 0, 48),$$

$$W^* = 35(4) + 81 = 221 = z^{\text{LP}} \quad (\text{ισχυρή δυϊκότητα} \checkmark)$$

Οικονομική ερμηνεία: $u^* = 4$ είναι η «τιμή» μιας μονάδας χωρητικότητας (ίση με τον λόγο p/w του οριακού — κλασματικού — αντικειμένου), και v_i είναι το πλεόνασμα αξίας του αντικειμένου i πάνω από το κόστος του χώρου που καταλαμβάνει.

7.3 (γ) Ακέραιο δυϊκό και έλεγχος της CS

Επιλύοντας το δυϊκό με $u, v_i \in \mathbb{Z}_{\geq 0}$ (CBC μέσω PuLP) προκύπτει η ίδια λύση, αφού ήταν ήδη ακέραια:

$$u^* = 4, \quad v^* = (0, 0, 0, 3, 30, 0, 48), \quad W_{\alpha\kappa}^* = 221.$$

(Πράγματι, θέτοντας $v_i = \max\{0, p_i - w_i u\}$ έχουμε $W(u) = 35u + \sum_i v_i$ με $W(3) = 223$, $W(4) = 221$, $W(5) = 222$ — το ελάχιστο είναι στο $u = 4$.)

i	$w_i u^*$	v_i^*	άθροισμα	p_i	περίσσεια	x_i^* (ακέραιο)
1	12	0	12	12	0	0
2	16	0	16	12	4	0
3	12	0	12	9	3	0
4	12	3	15	15	0	1
5	60	30	90	90	0	1
6	52	0	52	26	26	0
7	64	48	112	112	0	1

Έλεγχος των σχέσεων CS για το ακέραιο ζεύγος.

- (16) $x_i > 0 \Rightarrow w_i u + v_i = p_i$: για $i = 4, 5, 7$ η περίσσεια είναι 0 ✓
- (15) $v_i > 0 \Rightarrow x_i = 1$: τα $v_4, v_5, v_7 > 0$ αντιστοιχούν σε $x_4 = x_5 = x_7 = 1$ ✓

- (14) $u^* = 4 > 0 \Rightarrow \sum_i w_i x_i = 35$: στην πραγματικότητα $\sum_i w_i x_i^* = 3 + 15 + 16 = 34 < 35 \times$
ΠΑΡΑΒΙΑΖΕΤΑΙ

Ερμηνεία — γιατί δεν είναι σφάλμα. Οι σχέσεις συμπληρωματικής χαλαρότητας είναι *ισοδύναμες* με τη βελτιστότητα μόνο για το *συνεχές* ζεύγος πρωτεύοντος–δύϊκού, όπου ισχύει ισχυρή δuality. Στο ακέραιο πρόβλημα έχουμε

$$z_{\alpha\kappa}^* = 217 < 221 = W_{\alpha\kappa}^*, \quad \text{κενό ακεραιότητας} = 4.$$

Αφού $z_{\alpha\kappa}^* \neq W_{\alpha\kappa}^*$, είναι *αδύνατο* να ισχύουν ταυτόχρονα όλες οι σχέσεις CS — η ταυτόχρονη ισχύς τους θα συνεπαγόταν ισότητα των δύο αντικειμενικών. Εδώ η παραβίαση εντοπίζεται στη σχέση της χωρητικότητας: το σακίδιο δεν γεμίζει ακριβώς ($34 < 35$) γιατί κανένας ακέραιος συνδυασμός δεν αθροίζει σε 35 με μεγαλύτερη αξία, ενώ η συνεχής χαλάρωση εκμεταλλεύεται το $\frac{1}{3}$ του αντικειμένου 1 για να γεμίσει τη χωρητικότητα πλήρως.

Α' Παράρτημα: πηγαίος κώδικας

Όλος ο κώδικας είναι σε Python 3. Τα tableaux (Ασκήσεις 1 και 4) υπολογίζονται σε *ακριβή ρητή αριθμητική* (`fractions.Fraction`), ενώ οι επιλύσεις ΓΠ/ΑΓΠ γίνονται με PuLP (επιλυτής CBC) και `scipy.optimize`. Οι αλγόριθμοι Branch & Bound (Ασκήσεις 5 και 7) είναι *υλοποιημένοι ρητά* — ο solver καλείται μόνο για τα χαλαρωμένα ΓΠ των κόμβων — ώστε να φαίνεται το δέντρο και τα κριτήρια αποκοπής.

Α'.1 `ex_1.py` — ανάλυση ευαισθησίας με πλήρη tableaux

```

1  """Εργασία
2  2 - Άσκηση 1Ανάλυση
3  ευαισθησίας από το βέλτιστο tableau του
4
5   $max\ z = 2x_1 + x_2 + 4x_3 - x_4$ 
6  (P1)  $x_1 + 2x_2 + x_3 - 3x_4 \leq 8$ 
7  (P2)  $-x_2 + x_3 + 2x_4 \leq 0$ 
8  (P3)  $2x_1 + 7x_2 - 5x_3 - 10x_4 \leq 21, \quad x \geq 0$ 
9
10 κώδικας:
11 1. ανακατασκευάζει τον βασικό πίνακα B, τον  $B^{-1}$  και όλο το tableau,
12 2. για κάθε διαταραχή  $a(i)b(-)$  ελέγχει ΒΕΛΤΙΣΤΟΤΗΤΑ γραμμή(  $c_j - z_j$ )
13   και ΕΦΙΚΤΟΤΗΤΑ ( $B^{-1}b$ ) και επιλέγει primal ή dual simplex,
14 3. εκτελεί και ΤΥΠΩΝΕΙ όλες τις επαναλήψεις με ακριβή ρητή αριθμητική,
15 4. επαληθεύει κάθε αποτέλεσμα με ανεξάρτητη επίλυση (scipy.linprog).
16 """
17 from fractions import Fraction as F
18 import numpy as np
19 from scipy.optimize import linprog
20
21 nm = ["x1", "x2", "x3", "x4", "x5", "x6", "x7"]
22 A0 = [[F(v) for v in r] for r in
23        [[1, 2, 1, -3, 1, 0, 0], [0, -1, 1, 2, 0, 1, 0], [2, 7, -5, -10, 0, 0, 1]]]
24 m = 3
25 OPT_BASIS = [0, 3, 6] # x1, x4, x7
26
27
28 def build(basis, c, b):
29     """Πλήρες tableau + γραμμή ( $c_j - z_j$ ) + τιμή z, με ακριβή αριθμητική."""
30     T = [[A0[i][j] for j in range(7)] + [F(b[i])] for i in range(m)]
31     for r, bi in enumerate(basis):
32         p = next(k for k in range(r, m) if T[k][bi] != 0)
33         T[r], T[p] = T[p], T[r]
34         pv = T[r][bi]
35         T[r] = [v / pv for v in T[r]]
36         for k in range(m):
37             if k != r and T[k][bi] != 0:
38                 f = T[k][bi]
39                 T[k] = [T[k][t] - f * T[r][t] for t in range(8)]
40     cc = [F(v) for v in c] + [F(0)] * 3
41     red = [cc[j] - sum(cc[basis[i]] * T[i][j] for i in range(m)) for j in range(7)]
42     z = sum(cc[basis[i]] * T[i][7] for i in range(m))
43     return T, red, z
44
45
46 def show(tag, basis, T, red, z):
47     print(f"\n--- {tag} Βάση = ({', '.join(nm[j] for j in basis)}), z = {z}")
48     print(" " + " ".join(f"{v:>8}" for v in nm) + f"{'RHS':>8}")
49     for i in range(m):
50         print(f" {nm[basis[i]]:<6}" + " ".join(f"{str(T[i][j]):>8}" for j in range(7))
51               + f"{str(T[i][7]):>8}")
52     print(f" c-z " + " ".join(f"{str(red[j]):>8}" for j in range(7)) + f"{str(z):>8}")

```

```

53
54
55 def primal(basis, c, b, tag):
56     it = 0
57     while True:
58         T, red, z = build(basis, c, b)
59         show(f"{tag} - primal simplex, επανάληψη {it}", basis, T, red, z)
60         cand = [j for j in range(7) if j not in basis and red[j] > 0]
61         if not cand:
62             print(" => όλα τα  $c_j - z_j \leq 0$  : ΒΕΛΤΙΣΤΟ")
63             x = [F(0)] * 7
64             for i, bi in enumerate(basis):
65                 x[bi] = T[i][7]
66             print(f" x* = ({', '.join(str(x[j]) for j in range(4))}), z* = {z}")
67             return basis, z
68         e = max(cand, key=lambda j: red[j])
69         rat = [(T[i][7] / T[i][e], i) for i in range(m) if T[i][e] > 0]
70         mn = min(r for r, _ in rat)
71         i0 = [i for r, i in rat if r == mn][0]
72         print(f" εισέρχεται {nm[e]} (c-z = {red[e]}); λόγος: "
73               + ", ".join(f"{nm[basis[i]]}: {T[i][7]}/{T[i][e]} = {T[i][7]/T[i][e]}"
74                             for _, i in rat)
75               + f" -> εξέρχεται {nm[basis[i0]]}"
76               + (" ΕΦΥΛΙΣΜΕΝΟ( pivot, λόγος 0)" if mn == 0 else ""))
77         basis = basis[:i0] + [e] + basis[i0 + 1:]
78         it += 1
79
80
81 def dual(basis, c, b, tag):
82     it = 0
83     while True:
84         T, red, z = build(basis, c, b)
85         show(f"{tag} - dual simplex, επανάληψη {it}", basis, T, red, z)
86         neg = [i for i in range(m) if T[i][7] < 0]
87         if not neg:
88             print(" => όλα τα RHS  $\geq 0$  : ΕΦΙΚΤΟ και ΒΕΛΤΙΣΤΟ")
89             x = [F(0)] * 7
90             for i, bi in enumerate(basis):
91                 x[bi] = T[i][7]
92             print(f" x* = ({', '.join(str(x[j]) for j in range(4))}), z* = {z}")
93             return basis, z
94         r = min(neg, key=lambda i: T[i][7])
95         cand = [j for j in range(7) if j not in basis and T[r][j] < 0]
96         if not cand:
97             print(f" => η γραμμή {nm[basis[r]]} έχει RHS < 0 και ΟΛΑ τα στοιχεία  $\geq 0$ ")
98             print(" δεν υπάρχει επιτρεπτό pivot -> ΤΟ ΠΡΟΒΛΗΜΑ ΕΙΝΑΙ ΜΗ ΕΦΙΚΤΟ")
99             return None, None
100         e = min(cand, key=lambda j: abs(red[j] / T[r][j]))
101         print(f" εξέρχεται {nm[basis[r]]} (RHS = {T[r][7]}); λόγος |(c-z)/a|: "
102               + ", ".join(f"{nm[j]}: {abs(red[j]/T[r][j])}" for j in cand)
103               + f" -> εισέρχεται {nm[e]}")
104         basis = basis[:r] + [e] + basis[r + 1:]
105         it += 1
106
107
108 def check(c, b, tag):
109     r = linprog([-v for v in c], A_ub=[[float(x) for x in row[:4]] for row in A0],
110               b_ub=[float(v) for v in b], bounds=[(0, None)] * 4)
111     if r.status == 0:
112         print(f" [scipy] {tag}: x = {np.round(r.x, 6)}, z = {-r.fun:.6f}")
113     else:
114         print(f" [scipy] {tag}: {r.message}")
115
116

```

```

117 # ----- αρχικό tableau
118 c0, b0 = [2, 1, 4, -1], [8, 0, 21]
119 T, red, z = build(OPT_BASIS, c0, b0)
120 show("ΔΟΣΜΕΝΟ ΒΕΛΤΙΣΤΟ TABLEAU", OPT_BASIS, T, red, z)
121 Bmat = np.array([[float(A0[i][j]) for j in OPT_BASIS] for i in range(m)])
122 print("\n B =\n", Bmat)
123 print(" B^-1 =\n", np.linalg.inv(Bmat), 6))
124 print(" B^-1 b =", np.round(np.linalg.inv(Bmat) @ np.array(b0, float), 6))
125 print(" σκιάσεις τιμές y = cB B^-1 =",
126       np.round(np.array([2., -1., 0.] @ np.linalg.inv(Bmat), 6))
127
128 for tag, c, b, kind in [("α() c1 = 1", [1, 1, 4, -1], b0, "p"),
129                        ("β() c = [1,2,3,4]", [1, 2, 3, 4], b0, "p"),
130                        ("γ() b3 = 11", c0, [8, 0, 11], "d"),
131                        ("δ() b = [3,-2,4]", c0, [3, -2, 4], "d")]:
132     print("\n" + "#" * 78)
133     print("#" + tag)
134     print("#" * 78)
135     (primal if kind == "p" else dual)(list(OPT_BASIS), c, b, tag)
136     check(c, b, tag)

```

Α.2 ex_2.py — ανάθεση εργασιών, δυϊκές τιμές, σύγκριση με ακέραιο

```

1 """Εργασία
2 2 - Άσκηση 2Ανάθεση
3 4 εργασιών ABΓΔ(,,,) σε 3 εργαστήρια με δυνατότητα κλασματικήςεκτέλεσης
4 , με ελάχιστο συνολικό κόστος.Μεταβλητές
5
6 : x_ij = ποσοστό κλάσμα() της εργασίας j στο εργαστήριο iΚόστος
7 : c_ij = t_ij * k_i ώρες( * κόστοςώρα/)
8
9 min Σ_i Σ_j c_ij x_ij
10 s.t. Σ_i x_ij = 1 για κάθε εργασία j πλήρης( εκτέλεση)
11 Σ_j t_ij x_ij <= 160 για κάθε εργαστήριο i διαθέσιμος( χρόνος)
12 0 <= x_ij <= 1Επιπλέον
13
14 υπολογίζονται οι δυϊκές τιμές σκιάσεις( τιμές του χρόνου κάθεεργαστηρίου
15 και τιμή" κάθε εργασίας) και επαληθεύεται η ισχυρή δυϊκότητα.
16 """
17 import pulp
18
19 LABS = [1, 2, 3]
20 JOBS = ["A", "B", "Γ", "Δ"]
21 t = {(1, "A"): 32, (1, "B"): 151, (1, "Γ"): 72, (1, "Δ"): 118,
22      (2, "A"): 39, (2, "B"): 147, (2, "Γ"): 61, (2, "Δ"): 126,
23      (3, "A"): 46, (3, "B"): 155, (3, "Γ"): 57, (3, "Δ"): 121}
24 k = {1: 89, 2: 81, 3: 84}
25 CAP = {i: 160 for i in LABS}
26 c = {(i, j): t[i, j] * k[i] for i in LABS for j in JOBS}
27
28 print("Πίνακας κόστους c_ij = t_ij * k_i ευρώ( για ΟΛΟΚΛΗΡΗ την εργασία):")
29 print(" " + "".join(f"{j:>10}" for j in JOBS))
30 for i in LABS:
31     print(f"Εργ.{i} " + "".join(f"{c[i,j]:>10}" for j in JOBS))
32
33 # ----- μοντέλο
34 prob = pulp.LpProblem("anathesi_ergasiwn", pulp.LpMinimize)
35 x = pulp.LpVariable.dicts("x", (LABS, JOBS), lowBound=0, upBound=1, cat="Continuous")
36 prob += pulp.lpSum(c[i, j] * x[i][j] for i in LABS for j in JOBS), "Synoliko_Kostos"
37 for j in JOBS:
38     prob += pulp.lpSum(x[i][j] for i in LABS) == 1, f"ekteshb_{j}"
39 for i in LABS:

```

```

40     prob += pulp.lpSum(t[i, j] * x[i][j] for j in JOBS) <= CAP[i], f"xronos_{i}"
41
42 prob.solve(pulp.PULP_CBC_CMD(msg=0))
43 print(f"\nΚατάστασηn: {pulp.LpStatus[prob.status]}")
44 print(f"Ελάχιστο συνολικό κόστος Z* = {pulp.value(prob.objective):.4f} ευρώ\n")
45
46 print("Βέλτιστη ανάθεση x_ij:")
47 print("      " + "".join(f"{j:>10}" for j in JOBS) + f"ώρες{'':>10}")
48 for i in LABS:
49     hrs = sum(t[i, j] * x[i][j].value() for j in JOBS)
50     print(f"  Εργ_{i}  " + "".join(f"{x[i][j].value():>10.6f}" for j in JOBS)
51           + f"{hrs:>10.4f}")
52
53 print("\nΔιϊκέςn τιμές σκιάδεις( τιμές):")
54 for name, cons in prob.constraints.items():
55     print(f"  {name:<14} dual = {cons.pi:>12.4f}  slack = {cons.slack}")
56
57 # ----- έλεγχος ισχυρής διϊκότητας
58 u = {j: prob.constraints[f"ekteleshb_{j}"].pi for j in JOBS} # ελεύθερες
59 w = {i: prob.constraints[f"xronos_{i}"].pi for i in LABS}    # <= 0 (min)
60 W = sum(u[j] for j in JOBS) + sum(CAP[i] * w[i] for i in LABS)
61 print(f"\nΤιμήn διϊκής αντικειμενικής = {W:.4f}  ισούται( με Z* -> ισχυρή διϊκότητα)")
62
63 # ----- σύγκριση με ακέραια«» ανάθεση κάθε( εργασία
64 # ολόκληρη σε ένα εργαστήριο) ώστε να φανεί η αξία της κλασματικής εκτέλεσης
65 probI = pulp.LpProblem("anathesi_akeraia", pulp.LpMinimize)
66 y = pulp.LpVariable.dicts("y", (LABS, JOBS), cat="Binary")
67 probI += pulp.lpSum(c[i, j] * y[i][j] for i in LABS for j in JOBS)
68 for j in JOBS:
69     probI += pulp.lpSum(y[i][j] for i in LABS) == 1
70 for i in LABS:
71     probI += pulp.lpSum(t[i, j] * y[i][j] for j in JOBS) <= CAP[i]
72 probI.solve(pulp.PULP_CBC_CMD(msg=0))
73 print(f"\nΓιαn σύγκριση - αν ΔΕΝ επιτρεπόταν κλασματική εκτέλεση:")
74 print(f"  κατάσταση: {pulp.LpStatus[probI.status]}, κόστος = "
75       + f"{pulp.value(probI.objective):.2f} ευρώ  "
76       + f"διαφορά( +{pulp.value(probI.objective)-pulp.value(prob.objective):.2f})")
77 for j in JOBS:
78     for i in LABS:
79         if y[i][j].value() > 0.5:
80             print(f"      εργασία {j} -> Εργαστήριο {i}")

```

Α.3 ex_3.py — διϊκό και συμπληρωματική χαλαρότητα

```

1  """Εργασία
2  2 - Άσκηση 3Πρωτεύον
3  :
4      min z = 32x2 + 50x3 - 19x5
5      (P1) 15x1 + 15x2 + 15x3 + x5 = 40      -> y1 ελεύθερη
6      (P2) 12x1 - 90x2 + 14x4      >= 18      -> y2 >= 0
7      (P3)           x4 <= 11      -> y3 <= 0
8      xj >= 0Διϊκό
9
10 (max), ένας περιορισμός ανά πρωτεύουσα μεταβλητή, όλοι <= :
11     max W = 40y1 + 18y2 + 11y3
12     (x1) 15y1 + 12y2      <= 0
13     (x2) 15y1 - 90y2      <= 32
14     (x3) 15y1           <= 50
15     (x4)           14y2 + y3 <= 0
16     (x5) y1           <= -19α
17
18 () επίλυση διϊκού με PuLP/CBC + γεωμετρική περιγραφή της κορυφήςβ

```

```

19  () εύρεση της συμπληρωματικής λύσης του πρωτεύοντος MONO από τη δυϊκή
20  θεωρία (complementary slackness) – χωρίς να λυθεί το πρωτεύον.
21  """
22  import pulp
23
24  # =====
25  # α() ΔΥΪΚΟ
26  # =====
27  dual = pulp.LpProblem("dyiko", pulp.LpMaximize)
28  y1 = pulp.LpVariable("y1", lowBound=None, upBound=None) # ελεύθερη
29  y2 = pulp.LpVariable("y2", lowBound=0)                 # >= 0
30  y3 = pulp.LpVariable("y3", upBound=0)                  # <= 0
31  dual += 40 * y1 + 18 * y2 + 11 * y3, "W"
32  dual += 15 * y1 + 12 * y2 <= 0, "D_x1"
33  dual += 15 * y1 - 90 * y2 <= 32, "D_x2"
34  dual += 15 * y1 <= 50, "D_x3"
35  dual += 14 * y2 + y3 <= 0, "D_x4"
36  dual += y1 <= -19, "D_x5"
37  dual.solve(pulp.PULP_CBC_CMD(msg=0))
38
39  Y = {"y1": y1.value(), "y2": y2.value(), "y3": y3.value()}
40  print("=== α() ΔΥΪΚΟ ===")
41  print(f" κατάσταση: {pulp.LpStatus[dual.status]}")
42  print(f" y* = ({Y['y1']}, {Y['y2']}, {Y['y3']}), W* = {pulp.value(dual.objective)}")
43
44  DUALC = {"x1": (15 * Y["y1"] + 12 * Y["y2"], 0),
45           "x2": (15 * Y["y1"] - 90 * Y["y2"], 32),
46           "x3": (15 * Y["y1"], 50),
47           "x4": (14 * Y["y2"] + Y["y3"], 0),
48           "x5": (Y["y1"], -19)}
49  print("\n Δεσμευτικότητα δυϊκών περιορισμών:")
50  for kk, (lhs, rhs) in DUALC.items():
51      st = "ΔΕΣΜΕΥΤΙΚΟΣ" if abs(lhs - rhs) < 1e-9 else "μη δεσμευτικός"
52      print(f"    ({kk}) LHS = {lhs:.2f} RHS = {rhs:.2f} περίσσεια = "
53            f"{rhs-lhs:.2f} {st}")
54  print("\n Ενεργά υπερεπίπεδα στον R^3 στο σημείο y* = (-19, 0, 0):")
55  print("    (x5): y1 = -19 ; (x4): 14y2 + y3 = 0 ; όριο y2 >= 0 ; όριο y3 <= 0")
56  print("    -> 4 ενεργά υπερεπίπεδα ενώ αρκούν 3 => Η ΔΥΪΚΗ ΚΟΡΥΦΗ ΕΙΝΑΙ ΕΚΦΥΛΙΣΜΕΝΗ")
57
58  # απόδειξη μοναδικότητας του δυϊκού βέλτιστου αλγεβρικά():
59  print("\n Μοναδικότητα: από (x5) y1 <= -19 => 40y1 <= -760.")
60  print("    Από (x1) 12y2 <= -15y1 και από (x4) y3 <= -14y2, άρα")
61  print("    W = 40y1 + 18y2 + 11y3 <= 40y1 + 18y2 - 154y2 = 40y1 - 136y2 <= -760,")
62  print("    με ισότητα μόνο για y1 = -19, y2 = 0, y3 = 0. Άρα το δυϊκό βέλτιστο είναι ΜΟΝΑΔΙΚΟ.")
63
64  # =====
65  # β() ΣΥΜΠΛΗΡΩΜΑΤΙΚΗ ΧΑΛΑΡΟΤΗΤΑ -> ΛΥΣΗ ΠΡΩΤΕΥΟΝΤΟΣ
66  # =====
67  print("\n=== β() ΣΥΜΠΛΗΡΩΜΑΤΙΚΗ ΧΑΛΑΡΟΤΗΤΑ ===")
68  print(" CS-1: δυϊκός περιορισμός j ΜΗ δεσμευτικός => x_j = 0")
69  zero = [kk for kk, (l, r) in DUALC.items() if abs(l - r) > 1e-9]
70  print(f"    μη δεσμευτικοί: {zero} => " + ", ".join(f"{v}=0" for v in zero))
71  print(" CS-2: y_i != 0 => ο πρωτεύων περιορισμός i είναι δεσμευτικός")
72  print("    y1 = -19 != 0 -> ο P1 δεσμευτικός ισχύει( ούτως ή άλλως, είναι ισότητα)")
73  print("    y2 = 0, y3 = 0 -> δεν δίνουν πληροφορία για P2, P3")
74  print("\n Αντικατάσταση x1 = x2 = x3 = 0 στον P1: 15*0 + x5 = 40 => x5 = 40")
75  print(" 0 P2 απαιτεί 14 x4 >= 18 => x4 >= 9/7 ≈ 1.2857")
76  print(" 0 P3 απαιτεί x4 <= 11")
77  print(" Η αντικειμενική ΔΕΝ περιέχει x4 (c4 = 0) => κάθε x4 στο [9/7, 11] είναι βέλτιστο.")
78  z = 32 * 0 + 50 * 0 - 19 * 40
79  print(f"\n z* = -19 * 40 = {z} = W* -> ισχυρή δυϊκότητα ✓")
80  print(" Το σύνολο των βέλτιστων λύσεων του πρωτεύοντος είναι η ΑΚΜΗ")
81  print("    { (0,0,0,x4,40) : 9/7 <= x4 <= 11 }")
82  print(" με ΚΟΡΥΦΕΣ βασικές( λύσεις):")

```

```

83 print("    x = (0,0,0, 9/7, 40) δεσμευτικός( ο P2)")
84 print("    x = (0,0,0, 11, 40) δεσμευτικός( ο P3)")
85
86 # ----- ανεξάρτητη επαλήθευση
87 prim = pulp.LpProblem("prwteuon", pulp.LpMinimize)
88 X = [pulp.LpVariable(f"x{i+1}", lowBound=0) for i in range(5)]
89 prim += 32 * X[1] + 50 * X[2] - 19 * X[4]
90 prim += 15 * X[0] + 15 * X[1] + 15 * X[2] + X[4] == 40, "P1"
91 prim += 12 * X[0] - 90 * X[1] + 14 * X[3] >= 18, "P2"
92 prim += X[3] <= 11, "P3"
93 prim.solve(pulp.PULP_CBC_CMD(msg=0))
94 print(f"\n επαλήθευση[ PuLP] πρωτεύον: x = {[v.value() for v in X]}, "
95       f"z = {pulp.value(prim.objective)}")
96 print(" επαλήθευση[] και οι δύο κορυφές δίνουν z = -760:")
97 for x4 in (9/7, 11):
98     print(f"    x4 = {x4:.4f}: P2 = {14*x4:.3f} >= 18 ->"
99           f" {'OK' if 14*x4 >= 18-1e-9 else OXI''}, z = {-19*x4}")

```

A.4 ex_4.py — dual simplex με tableaux

```

1  """Εργασία
2  2 - Άσκηση 4Επίλυση
3  με τον αλγόριθμο DUAL SIMPLEX του
4
5  min z = 3x1 + 4x2 + 6x3 + 7x4 + x5
6  (P1) 2x1 - x2 + x3 + 6x4 - 5x5 >= 6
7  (P2) x1 + x2 + 2x3 + x4 + 2x5 >= 3 , x >= 0Πολλαπλασιάζουμε
8
9  τους περιορισμούς με -1 και βάζουμε slack s1, s2:η
10 αρχική βάση {s1, s2} είναι ΔΥΪΚΑ εφικτή όλα( τα c_j >= 0 σε min) αλλάΠΡΩΤΕΥΟΝΤΩΣ
11 μη εφικτή (RHS < 0) - ιδανική εκκίνηση για dual simplex.0
12
13 κώδικας τυπώνει κάθε επανάληψη σε μορφή tableau με ακριβή ρητήαριθμητική
14 , και στο τέλος ελέγχει εκφυλισμό, μοναδικότητα της βέλτιστηςλύσης
15 και το σύνολο των βέλτιστων δυϊκών λύσεων.
16 """
17 from fractions import Fraction as F
18 import numpy as np
19 from scipy.optimize import linprog
20
21 nm = ["x1", "x2", "x3", "x4", "x5", "s1", "s2"]
22 A = [[F(v) for v in r] for r in [[-2, 1, -1, -6, 5, 1, 0],
23                                 [-1, -1, -2, -1, -2, 0, 1]]]
24 b = [F(-6), F(-3)]
25 c = [F(v) for v in [3, 4, 6, 7, 1, 0, 0]]
26 m = 2
27
28
29 def build(basis):
30     T = [A[i][:] + [b[i]] for i in range(m)]
31     for r, bi in enumerate(basis):
32         p = next(k for k in range(r, m) if T[k][bi] != 0)
33         T[r], T[p] = T[p], T[r]
34         pv = T[r][bi]
35         T[r] = [v / pv for v in T[r]]
36         for k in range(m):
37             if k != r and T[k][bi] != 0:
38                 f = T[k][bi]
39                 T[k] = [T[k][t] - f * T[r][t] for t in range(8)]
40     red = [c[j] - sum(c[basis[i]] * T[i][j] for i in range(m)) for j in range(7)]
41     z = sum(c[basis[i]] * T[i][7] for i in range(m))
42     return T, red, z

```

```

43
44
45 def show(tag, basis, T, red, z):
46     print(f"\n--- {tag}   Βάση = ({', '.join(nm[j] for j in basis)}),   z = {z}")
47     print("         " + "".join(f"{v:>9}" for v in nm) + f"{'RHS':>9}")
48     for i in range(m):
49         print(f"      {nm[basis[i]]:<6}" + "".join(f"{str(T[i][j]):>9}" for j in range(7))
50               + f"{'str(T[i][7]):>9}")
51     print(" c-z      " + "".join(f"{str(red[j]):>9}" for j in range(7)) + f"{'str(z):>9}")
52
53
54 basis = [5, 6]
55 it = 0
56 print("=" * 82)
57 print("DUAL SIMPLEX")
58 print("=" * 82)
59 while True:
60     T, red, z = build(basis)
61     show(f"Επανάληψη {it}", basis, T, red, z)
62     neg = [i for i in range(m) if T[i][7] < 0]
63     if not neg:
64         print("\n Όλα τα RHS >= 0 πρωτεύουσα( εφικτότητα) και όλα τα c_j - z_j >= 0")
65         print("   δυϊκή( εφικτότητα)  =>  ΒΕΛΤΙΣΤΟ.")
66         break
67     r = min(neg, key=lambda i: T[i][7])
68     print(f"   εξερχόμενη: {nm[basis[r]]} πιο( αρνητικό RHS = {T[r][7]})")
69     cand = [j for j in range(7) if j not in basis and T[r][j] < 0]
70     if not cand:
71         print("   δεν υπάρχει αρνητικό στοιχείο στη γραμμή -> ΜΗ ΕΦΙΚΤΟ")
72         break
73     print("   ratio test |(c_j - z_j)/a_rj| μόνο για a_rj < 0: "
74           + ", ".join(f"{nm[j]}: {red[j]}/{T[r][j]} = {abs(red[j]/T[r][j])}" for j in cand))
75     e = min(cand, key=lambda j: abs(red[j] / T[r][j]))
76     print(f"   -> εισερχόμενη: {nm[e]},   στοιχείο οδηγός = {T[r][e]}")
77     basis = basis[:r] + [e] + basis[r + 1:]
78     it += 1
79
80 T, red, z = build(basis)
81 x = [F(0)] * 7
82 for i, bi in enumerate(basis):
83     x[bi] = T[i][7]
84 print(f"\n  ΒΕΛΤΙΣΤΗ ΛΥΣΗ: x* = ({', '.join(str(x[j]) for j in range(5))}),   z* = {z}")
85 print(f"   P1: {2*x[0]-x[1]+x[2]+6*x[3]-5*x[4]} >= 6 ,   "
86       f"P2: {x[0]+x[1]+2*x[2]+x[3]+2*x[4]} >= 3   και( οι δύο δεσμευτικοί)")
87
88 # ----- εκφυλισμός / μοναδικότητα
89 zb = [nm[basis[i]] for i in range(m) if T[i][7] == 0]
90 print(f"\n  ΕΚΦΥΛΙΣΜΟΣ: βασικές μεταβλητές με τιμή 0: {zb}")
91 print("   Στον R^5 μια κορυφή ορίζεται από 5 ενεργά υπερπίεδα· εδώ είναι ενεργά")
92 print("   τα x2=0, x3=0, x4=0, x5=0 ΚΑΙ οι δύο περιορισμοί P1, P2 -> 6 > 5,")
93 print("   άρα η βέλτιστη κορυφή είναι ΕΚΦΥΛΙΣΜΕΝΗ.")
94 nb = [j for j in range(7) if j not in basis]
95 print(f"   ΜΟΝΑΔΙΚΟΤΗΤΑ: c_j - z_j των μη βασικών = "
96       f"{'{nm[j]: str(red[j]) for j in nb} }")
97 print("   όλα ΓΝΗΣΙΩΣ θετικά => η βέλτιστη λύση είναι ΜΟΝΑΔΙΚΗ.")
98
99 # ----- δυϊκό & επαλήθευση
100 print(f"\n  ΔΥΪΚΟ: max 6u1 + 3u2   s.t.   2u1+u2<=3, -u1+u2<=4, u1+2u2<=6,")
101 print("          6u1+u2<=7, -5u1+2u2<=1,   u >= 0")
102 print("   Λόγω του εκφυλισμού της πρωτεύουσας κορυφής, το δυϊκό έχει ΠΟΛΛΑΠΛΕΣ")
103 print("   βέλτιστες λύσεις: για κάθε u1 στο [5/9, 1] με u2 = 3 - 2u1 ισχύει W = 9.")
104 for u1 in [F(5, 9), F(3, 4), F(1)]:
105     u2 = 3 - 2 * u1
106     ok = all(v <= 0 for v in [2*u1+u2-3, -u1+u2-4, u1+2*u2-6, 6*u1+u2-7, -5*u1+2*u2-1])

```

```

107     print(f"      u = ({u1}, {u2}) επικτική: {ok}, W = {6*u1+3*u2}")
108
109     r = linprog([3, 4, 6, 7, 1], A_ub=[[-2, 1, -1, -6, 5], [-1, -1, -2, -1, -2]],
110               b_ub=[-6, -3], bounds=[(0, None)] * 5)
111     print(f"\n   επαλήθευση[ scipy] x = {np.round(r.x,6)}, z = {r.fun:.6f}")

```

A.5 ex_5.py — Branch & Bound με ρητά κριτήρια αποκοπής

```

1  """Εργασία
2  2 - Άσκηση 5
3  Branch & Bound για το μικτό ακέραιο πρόβλημα
4
5      min z = 12x1 + 45x2 + 67x3 + x4
6      (P1) 4x1 + 2x2 - x4 <= 10
7      (P2) 6x1 + 19x3 >= 5
8           x2, x3, x4 >= 0, x1 ∈ {0,1}, x3 ακέραιος0
9
10  κώδικας υλοποιεί ρητά το B&B δεν( το αφήνει στον MILP solver): σε κάθεκόμβο
11  λύνει μόνο το ΧΑΛΑΡΩΜΕΝΟ LP με PuLP/CBC και εφαρμόζει τα τρίακριτήρια
12  αποκοπής (fathoming):
13  (i) μη επικτικότητα,
14  (ii) ακέραια λύση -> ενημέρωση incumbent,
15  (iii) φράγμα: zLP >= incumbent πρόβλημα( ΕΛΑΧΙΣΤΟΠΟΙΗΣΗΣ)Η
16  επιλογή του επόμενου κόμβου γίνεται με best-first μικρότερο( zLP).
17  """
18  import pulp
19
20  INT_VARS = ["x1", "x3"] # ακέραιες μεταβλητές
21  BOUNDS0 = {"x1": (0, 1), "x2": (0, None), "x3": (0, None), "x4": (0, None)}
22
23
24  def solve_lp(bnds):
25      """Χαλαρωμένο LP του κόμβου· επιστρέφει (status, x, z)."""
26      p = pulp.LpProblem("node", pulp.LpMinimize)
27      v = {k: pulp.LpVariable(k, lowBound=bnds[k][0], upBound=bnds[k][1])
28            for k in BOUNDS0}
29      p += 12 * v["x1"] + 45 * v["x2"] + 67 * v["x3"] + v["x4"]
30      p += 4 * v["x1"] + 2 * v["x2"] - v["x4"] <= 10
31      p += 6 * v["x1"] + 19 * v["x3"] >= 5
32      st = p.solve(pulp.PULP_CBC_CMD(msg=0))
33      if pulp.LpStatus[st] != "Optimal":
34          return pulp.LpStatus[st], None, None
35      return "Optimal", {k: v[k].value() for k in BOUNDS0}, pulp.value(p.objective)
36
37
38  def frac_var(x):
39      for k in INT_VARS:
40          if abs(x[k] - round(x[k])) > 1e-6:
41              return k
42      return None
43
44
45  counter = 0
46  incumbent, best_x = float("inf"), None
47  st, x, z = solve_lp(dict(BOUNDS0))
48  counter += 1
49  nodes = [(1, "ρίζα", None, st, x, z, "")]
50  # Λίστα ΖΩΝΤΑΝΩΝ κόμβων: (zLP, όρια, id). Η επιλογή γίνεται με best-first,
51  # δηλαδή παίρνουμε κάθε φορά τον κόμβο με το ΜΙΚΡΟΤΕΡΟ zLP ελαχιστοποίηση().
52  live = [(z, dict(BOUNDS0), 1)]
53  nid = 1
54  print("=" * 90)

```

```

55 print("BRANCH & BOUND (best-first, ελαχιστοποίηση)")
56 print("=" * 90)
57 print(f"κ1 ρίζα(): LP -> x = {x}, zLP = {z:.4f}")
58
59 while live:
60     best_node = min(live, key=lambda n: n[0])    # <-- best-first
61     live.remove(best_node)
62     z, bnds, myid = best_node
63     if z >= incumbent - 1e-9:
64         print(f" κ{myid}: ΑΠΟΚΟΠΗ από φράγμα (zLP = {z:.4f} >= incumbent = {incumbent:.4f})")
65         continue
66     st, x, _ = solve_lp(bnds)
67     j = frac_var(x)
68     print(f"\n Διακλάδωση του κ{myid} (zLP = {z:.4f}) στη μεταβλητή {j} = {x[j]:.4f}")
69     lo, hi = bnds[j]
70     for side, nb in (( "<=" + str(int(x[j] // 1)), (lo, int(x[j] // 1))),
71                     (">=" + str(int(x[j] // 1) + 1), (int(x[j] // 1) + 1, hi))):
72         nid += 1
73         child = dict(bnds)
74         child[j] = nb
75         if nb[1] is not None and nb[0] > nb[1]:
76             print(f" κ{nid}: {j} {side} -> ΚΕΝΟ ΔΙΑΣΤΗΜΑ μη( εφικτό)")
77             nodes.append((nid, f"{j} {side}", myid, "Infeasible", None, None, "μη εφικτό"))
78             continue
79         s2, x2, z2 = solve_lp(child)
80         counter += 1
81         if s2 != "Optimal":
82             print(f" κ{nid}: {j} {side} -> ΜΗ ΕΦΙΚΤΟ αποκοπή()")
83             nodes.append((nid, f"{j} {side}", myid, "Infeasible", None, None, "μη εφικτό"))
84             continue
85         f2 = frac_var(x2)
86         if z2 >= incumbent - 1e-9:
87             act = f"αποκοπή από φράγμα ({z2:.2f} >= {incumbent:.2f})"
88         elif f2 is None:
89             incumbent, best_x = z2, x2
90             act = f"ΑΚΕΡΑΙΑ -> νέο incumbent z = {z2:.2f}"
91         else:
92             act = f"ζωντανός κλασματικό( {f2})"
93             live.append((z2, child, nid))
94         print(f" κ{nid}: {j} {side} -> x = "
95               f"{ {k: round(vv,4) for k,vv in x2.items()} }, zLP = {z2:.4f} [{act}]")
96         nodes.append((nid, f"{j} {side}", myid, s2, x2, z2, act))
97
98 print("\n" + "=" * 90)
99 print(f"ΒΕΛΤΙΣΤΗ ΑΚΕΡΑΙΑ ΛΥΣΗ: x* = { {k: round(v,4) for k,v in best_x.items()} }")
100 print(f"z* = {incumbent:.4f} (LP φράγμα ρίζας = {nodes[0][5]:.4f}, "
101        f"κενό ακεραιότητας = {incumbent - nodes[0][5]:.4f})")
102 print(f"Συνολικοί κόμβοι που λύθηκαν: {counter}")
103
104 # ----- ανεξάρτητη επαλήθευση
105 p = pulp.LpProblem("check", pulp.LpMinimize)
106 x1 = pulp.LpVariable("x1", cat="Binary")
107 x2 = pulp.LpVariable("x2", lowBound=0)
108 x3 = pulp.LpVariable("x3", lowBound=0, cat="Integer")
109 x4 = pulp.LpVariable("x4", lowBound=0)
110 p += 12 * x1 + 45 * x2 + 67 * x3 + x4
111 p += 4 * x1 + 2 * x2 - x4 <= 10
112 p += 6 * x1 + 19 * x3 >= 5
113 p.solve(pulp.PULP_CBC_CMD(msg=0))
114 print(f"\nεπαλήθευση[ MILP solver] x = ({x1.value()}, {x2.value()}, {x3.value()}, "
115        f"{x4.value()}), z = {pulp.value(p.objective)}")

```

Α.6 ex_6.py — MIP με big-M και αναλυτική επαλήθευση

```

1  """Εργασία
2  2 – Άσκηση 6Διατύπωση
3  ως ΜΙΚΤΟ ΑΚΕΡΑΙΟ πρόγραμμα και επίλυση του
4
5   $\min f_1(x_1) + f_2(x_2), \quad f_1 = 5+4x_1 \ (x_1>0) \text{ ή } 0 \ (x_1=0)$ 
6   $f_2 = 3+5x_2 \ (x_2>0) \text{ ή } 0 \ (x_2=0)$ 
7
8  1)  $x_1 \geq 5 \ \vee \ x_2 \geq 5$ 
9  2) τουλάχιστον ένας από:  $2x_1+x_2 \geq 12$ ,  $x_1+x_2 \geq 10$ ,  $x_1+2x_2 \geq 12$ 
10 3)  $|x_1 - x_2| \in \{0, 5, 10\}$ 
11 4)  $x_1, x_2 \geq 0$  ΤΕΧΝΙΚΕΣ
12
13 :
14 * fixed cost :  $x_i \leq M y_i$ , όρος  $+F_i y_i$  στην αντικειμενική
15 * διάζευξη (OR) :  $z_1+z_2 \geq 1$ ,  $x_1 \geq 5 \implies z_1$ ,  $x_2 \geq 5 \implies z_2$ 
16 * at-least-one :  $w_1+w_2+w_3 \geq 1$  και  $g_r(x) \geq b_r - M(1-w_r)$ 
17 *  $|x_1-x_2| = v$  :  $v = 5 \implies a_5 + 10 \implies a_{10}$  με  $a_0+a_5+a_{10} = 1$  και δυαδική  $s$ 
18 για το πρόσημο ( $x_1-x_2 = v$  αν  $s=1$ ,  $x_2-x_1 = v$  αν  $s=0$ ) ΤΕΚΜΗΡΙΩΣΗ
19
20 ΤΟΥ  $M$ : κάθε βέλτιστη λύση έχει  $x_1, x_2 \leq 10$  βλ. ανάλυση περιπτώσεων
21 παρακάτω), άρα το φράγμα  $x_i \leq 20$  δεν αποκόπτει καμία βέλτιστη λύση
22 και  $M = 40$  είναι ασφαλές ( $\geq 2 \cdot 20$  ώστε να σβήνει) κάθε ανενεργό περιορισμό
23 ). Ο κώδικας επαληθεύει ότι το αποτέλεσμα ΔΕΝ εξαρτάται από το  $M$ .
24 """
25
26 import pulp
27
28 F1, V1 = 5, 4 # σταθερό και μεταβλητό κόστος της  $f_1$ 
29 F2, V2 = 3, 5 # σταθερό και μεταβλητό κόστος της  $f_2$ 
30 UB = 20 # ασφαλές άνω φράγμα για τα  $x_1, x_2$ 
31
32 def build(M, force_v=None):
33     p = pulp.LpProblem("mip", pulp.LpMinimize)
34     x1 = pulp.LpVariable("x1", 0, UB)
35     x2 = pulp.LpVariable("x2", 0, UB)
36     y1 = pulp.LpVariable("y1", cat="Binary")
37     y2 = pulp.LpVariable("y2", cat="Binary")
38     z1 = pulp.LpVariable("z1", cat="Binary")
39     z2 = pulp.LpVariable("z2", cat="Binary")
40     w = [pulp.LpVariable(f"w{r}", cat="Binary") for r in range(3)]
41     a0 = pulp.LpVariable("a0", cat="Binary")
42     a5 = pulp.LpVariable("a5", cat="Binary")
43     a10 = pulp.LpVariable("a10", cat="Binary")
44     s = pulp.LpVariable("s", cat="Binary")
45
46     p += F1 * y1 + V1 * x1 + F2 * y2 + V2 * x2 # γραμμικοποιημένη()
47
48     p += x1 <= UB * y1 # fixed costs
49     p += x2 <= UB * y2
50
51     p += z1 + z2 >= 1 # περιορισμός 1 (OR)
52     p += x1 >= 5 * z1
53     p += x2 >= 5 * z2
54
55     p += w[0] + w[1] + w[2] >= 1 # περιορισμός 2
56     p += 2 * x1 + x2 >= 12 - M * (1 - w[0])
57     p += x1 + x2 >= 10 - M * (1 - w[1])
58     p += x1 + 2 * x2 >= 12 - M * (1 - w[2])
59
60     p += a0 + a5 + a10 == 1 # περιορισμός 3
61     v = 5 * a5 + 10 * a10
62     if force_v is not None: # για την ανάλυση
63         p += a0 == (1 if force_v == 0 else 0)

```

```

63     p += a5 == (1 if force_v == 5 else 0)
64     p += a10 == (1 if force_v == 10 else 0)
65     p += x1 - x2 <= v + M * (1 - s)
66     p += x1 - x2 >= v - M * (1 - s)
67     p += x2 - x1 <= v + M * s
68     p += x2 - x1 >= v - M * s
69     return p, x1, x2, (y1, y2, z1, z2, w, a0, a5, a10, s)
70
71
72 print("=== Επίλυση του πλήρους MIP ===")
73 p, X1, X2, B = build(40)
74 p.solve(pulp.PULP_CBC_CMD(msg=0))
75 y1, y2, z1, z2, w, a0, a5, a10, s = B
76 print(f" κατάσταση: {pulp.LpStatus[p.status]}")
77 print(f" x1* = {X1.value():.6f} (= 17/3), x2* = {X2.value():.6f} (= 2/3)")
78 print(f" Z* = {pulp.value(p.objective):.6f}")
79 print(f" δυαδικές: y1={y1.value():.0f} y2={y2.value():.0f} | z1={z1.value():.0f} "
80       f"z2={z2.value():.0f} | w={[int(v.value()) for v in w]} | "
81       f"a0={a0.value():.0f} a5={a5.value():.0f} a10={a10.value():.0f} | s={s.value():.0f}")
82
83 x1v, x2v = X1.value(), X2.value()
84 print("\n Έλεγχος των APXΙΚΩΝ μη( γραμμικών) περιορισμών:")
85 print(f" 1) x1>=5 ή x2>=5 : {x1v:.4f} >= 5 -> {x1v >= 5 - 1e-6}")
86 print(f" 2) 2x1+x2 = {2*x1v+x2v:.4f} >= 12 -> {2*x1v+x2v >= 12-1e-6}"
87       f" | x1+x2 = {x1v+x2v:.4f} | x1+2x2 = {x1v+2*x2v:.4f}")
88 print(f" 3) |x1-x2| = {abs(x1v-x2v):.4f} ∈ {{0,5,10}} -> {abs(abs(x1v-x2v)-5) < 1e-6}")
89 print(f" 4) x1,x2 >= 0 -> True")
90 f1 = F1 + V1 * x1v if x1v > 1e-9 else 0
91 f2 = F2 + V2 * x2v if x2v > 1e-9 else 0
92 print(f" f1({x1v:.4f}) = {f1:.4f} , f2({x2v:.4f}) = {f2:.4f} , σύνολο = {f1+f2:.4f}")
93
94 print("\n=== Ανεξαρτησία από το M ευρωστιά ( της διατύπωσης) ===")
95 for M in (25, 40, 100, 1000):
96     pp, a, b, _ = build(M)
97     pp.solve(pulp.PULP_CBC_CMD(msg=0))
98     print(f" M = {M:>5}: Z* = {pulp.value(pp.objective):.6f}, "
99           f"x = ({a.value():.4f}, {b.value():.4f})")
100
101 print("\n=== Ανάλυση ανά τιμή του |x1 - x2| ===")
102 for v in (0, 5, 10):
103     pp, a, b, _ = build(40, force_v=v)
104     pp.solve(pulp.PULP_CBC_CMD(msg=0))
105     if pulp.LpStatus[pp.status] == "Optimal":
106         print(f" |x1-x2| = {v:>2}: x = ({a.value():.4f}, {b.value():.4f}), "
107               f"Z = {pulp.value(pp.objective):.4f}")
108     else:
109         print(f" |x1-x2| = {v:>2}: {pulp.LpStatus[pp.status]}")
110
111 print("\n=== Ανεξάρτητη επαλήθευση με πλήρη ανάλυση περιπτώσεων ===")
112 # Σε κάθε περίπτωση το πρόβλημα γίνεται διάστατο1- και λύνεται αναλυτικά.
113 best = (float("inf"), None)
114 cases = []
115 # x2 = 0 : τότε πρέπει x1 >= 5 και |x1| ∈ {0,5,10} -> x1 ∈ {5, 10}
116 for x1v in (5, 10):
117     ok2 = max(2*x1v, x1v, x1v) >= 12 if False else (2*x1v >= 12 or x1v >= 10 or x1v >= 12)
118     if ok2:
119         cases.append((x1v, 0, F1 + V1*x1v))
120 # x1 = 0 : x2 ∈ {5, 10}
121 for x2v in (5, 10):
122     if (x2v >= 12) or (x2v >= 10) or (2*x2v >= 12):
123         cases.append((0, x2v, F2 + V2*x2v))
124 # x1, x2 > 0 : για κάθε d ∈ {0,5,10}, κάθε πρόσημο και κάθε επιλογή του
125 # τουλάχιστον1 ενός2 περιορισμού της ομάδας 2, μένει ένα ΓΡΑΜΜΙΚΟ πρόβλημα
126 # μιας μεταβλητής, που λύνεται ακριβώς (Fraction).

```

```

127 from fractions import Fraction as Fr
128 G = [(2, 1, 12), (1, 1, 10), (1, 2, 12)]      # a*x1 + b*x2 >= c
129 for d in (0, 5, 10):
130     for sgn in (+1, -1):
131         for (a, b, cst) in G:
132             # x1 = t + d αν sgn=+1 αλλιώς x2 = t + d, με t >= 0
133             if sgn == 1:
134                 # a(t+d) + b t >= cst -> t >= (cst - a*d)/(a+b)
135                 t = max(Fr(0), Fr(cst - a*d, a + b))
136                 # και ο περιορισμός 1: x1 = t+d >= 5 ή x2 = t >= 5
137                 if not (t + d >= 5):
138                     t = max(t, Fr(5))
139                 x1v, x2v = t + d, t
140             else:
141                 t = max(Fr(0), Fr(cst - b*d, a + b))
142                 if not (t + d >= 5):
143                     t = max(t, Fr(5))
144                 x1v, x2v = t, t + d
145             if x1v <= 0 or x2v <= 0:
146                 continue
147             cases.append((float(x1v), float(x2v),
148                           float(F1 + V1*x1v + F2 + V2*x2v)))
149 for x1v, x2v, val in cases:
150     if val < best[0] - 1e-9:
151         best = (val, (x1v, x2v))
152 print(f" ελάχιστο από την ανάλυση περιπτώσεων: Z = {best[0]:.4f} στο "
153       f"x = ({best[1][0]:.4f}, {best[1][1]:.4f})")

```

A.7 ex_7.py — σακίδιο: best-first B&B, δυϊκό, CS

```

1  """Εργασία
2  2 - Άσκηση 7 πρόβλημα( σακιδίου 0-1)
3
4  max z = 12x1+12x2+9x3+15x4+90x5+26x6+112x7
5  s.t.  3x1+4x2+3x3+3x4+15x5+13x6+16x7 <= 35 ,  xi ∈ {0,1}α
6
7  () Branch & Bound με στρατηγική BEST-FIRST (jumptracking): από όλους τους
8  ζωντανούς κόμβους διακλαδίζεται κάθε φορά αυτός με το ΜΕΓΑΛΥΤΕΡΟ άνω
9  φράγμα zLP. Το χαλαρωμένο LP λύνεται με τον άπληστο«» αλγόριθμο κατά
10 φθίνοντα λόγο p_i/w_i βέλτιστος( για το κλασματικό knapsack).β
11 () Δυϊκό του χαλαρωμένου LP + σχέσεις συμπληρωματικής χαλαρότητας και
12 παραγωγή της δυϊκής λύσης από αυτές.γ
13 () Ακέραιο δυϊκό με solver και έλεγχος των σχέσεων CS για το ΑΚΕΡΑΙΟ ζεύγος.
14 """
15 from fractions import Fraction as F
16 import pulp
17
18 p = [12, 12, 9, 15, 90, 26, 112]
19 w = [3, 4, 3, 3, 15, 13, 16]
20 CAP = 35
21 n = 7
22 order = sorted(range(n), key=lambda i: (-F(p[i], w[i]), i))
23
24 print("Δεδομένα ταξινομημένα( κατά φθίνοντα λόγο p_i/w_i):")
25 print(f"  {'item':>6}{ 'p':>6}{ 'w':>5}{ 'p/w':>8}")
26 for i in order:
27     print(f"  {'x'+str(i+1):>6}{p[i]:>6}{w[i]:>5}{float(F(p[i],w[i])):>8.3f}")
28
29
30 def lp_bound(fix):
31     """Ακριβής λύση του χαλαρωμένου κλασματικού() knapsack με άπληστο κανόνα."""
32     cap, val = F(CAP), F(0)

```

```

33 x = [F(0)] * n
34 for i, vv in fix.items():
35     x[i] = F(vv)
36     cap -= w[i] * x[i]
37     val += p[i] * x[i]
38 if cap < 0:
39     return None, None
40 for i in order:
41     if i in fix:
42         continue
43     take = min(F(1), cap / w[i])
44     x[i] = take
45     cap -= w[i] * take
46     val += p[i] * take
47 return x, val
48
49
50 def fractional(x):
51     return [i for i in range(n) if x[i] not in (0, 1)]
52
53
54 # =====
55 # α() BEST-FIRST BRANCH & BOUND
56 # =====
57 print("\n" + "=" * 94)
58 print("α() BRANCH & BOUND - BEST FIRST")
59 print("=" * 94)
60 x0, z0 = lp_bound({})
61 nid = 1
62 rows = [(1, "ρίζα χωρίς( δεσμεύσεις)", None, x0, z0, "")]
63 # Λίστα ΖΩΝΤΑΝΩΝ κόμβων: (zLP, δεσμεύσεις, id). Best-first σημαίνει ότι
64 # επιλέγουμε κάθε φορά τον κόμβο με το ΜΕΓΑΛΥΤΕΡΟ άνω φράγμα zLP.
65 live = [(z0, {}, 1)]
66 incumbent, best_x, step = F(0), None, 0
67 seq = []
68 while live:
69     best_node = max(live, key=lambda n: n[0]) # <-- best-first / jumtracking
70     live.remove(best_node)
71     zb, fix, myid = best_node
72     step += 1
73     if zb <= incumbent:
74         seq.append((step, myid, f"αποκοπή από φράγμα ({zb} <= {incumbent})"))
75         for r in range(len(rows)):
76             if rows[r][0] == myid:
77                 rows[r] = rows[r][:5] + (f"αποκοπή από φράγμα ({zb} <= incumbent {incumbent})",)
78             continue
79     x, _ = lp_bound(fix)
80     fr = fractional(x)
81     j = fr[0]
82     seq.append((step, myid, f"διακλάδωση στο x{j+1} (zLP = {zb})"))
83     for r in range(len(rows)):
84         if rows[r][0] == myid:
85             rows[r] = rows[r][:5] + (f"διακλάδωση στο x{j+1}",)
86     for val in (0, 1):
87         nid += 1
88         f2 = dict(fix)
89         f2[j] = val
90         x2, z2 = lp_bound(f2)
91         if x2 is None:
92             rows.append((nid, f"x{j+1}={val}", myid, None, None, "MH ΕΦΙΚΤΟ"))
93             continue
94         if z2 <= incumbent:
95             act = f"αποκοπή από φράγμα ({z2} <= incumbent {incumbent})"
96         elif not fractional(x2):

```

```

97     incumbent, best_x = z2, x2
98     act = f"ΑΚΕΡΑΙΟ z = {z2} -> νέο incumbent"
99     else:
100         act = "ζώντανός"
101         live.append((z2, f2, nid))
102         rows.append((nid, f"x{j+1}={val}", myid, x2, z2, act))
103
104 print(f"\nk{'':>3} γονέας{'':>7} δέσμευση{'':<10} λύση{' LP':<34} {'zLP':>6} ενέργεια")
105 for r in rows:
106     xs = "(" + ",".join(str(v) for v in r[3]) + ")" if r[3] else "-"
107     zs = str(r[4]) if r[4] is not None else "-"
108     print(f"{r[0]:>3} {str(r[2]):>7} {r[1]:<10} {xs:<34} {zs:>6} {r[5]}")
109
110 print("\Σειράν επιλογής κόμβων (best-first):")
111 for s, i, a in seq:
112     print(f" βήμα {s}: κόμβος κ{i} -> {a}")
113
114 sel = [i + 1 for i in range(n) if best_x[i] == 1]
115 print(f"\ΒΕΛΤΙΣΤΗΝ ΑΚΕΡΑΙΑ ΛΥΣΗ: x* = ({','.join(str(v) for v in best_x)})")
116 print(f" αντικείμενα: {sel}, βάρος = {sum(w[i-1] for i in sel)} <= {CAP}, z* = {incumbent}")
117 print(f" άνω φράγμα ρίζας zLP = {z0} -> κενό ακεραιότητας = {z0 - incumbent}")
118 print(f" συνολικοί κόμβοι: {len(rows)}")
119
120 # =====
121 # β() ΔΥΪΚΟ ΤΟΥ ΧΑΛΑΡΩΜΕΝΟΥ ΚΑΙ CS
122 # =====
123 print("\n" + "=" * 94)
124 print("β() ΔΥΪΚΟ ΤΟΥ ΧΑΛΑΡΩΜΕΝΟΥ LP ΚΑΙ ΣΥΜΠΛΗΡΩΜΑΤΙΚΗ ΧΑΛΑΡΟΤΗΤΑ")
125 print("=" * 94)
126 print(" Πρωτεύον χαλαρωμένο(): max Σ p_i x_i, Σ w_i x_i <= 35 (u), x_i <= 1 (v_i), x_i >= 0")
127 print(" Δυϊκό: min W = 35u + Σ v_i s.t. w_i u + v_i >= p_i, u, v_i >= 0")
128 print("\n Σχέσεις CS:")
129 print(" (1) u > 0 => Σ w_i x_i = 35")
130 print(" (2) v_i > 0 => x_i = 1")
131 print(" (3) x_i > 0 => w_i u + v_i = p_i")
132 print(" (4) w_i u + v_i > p_i => x_i = 0")
133 print(" (5) x_i < 1 => v_i = 0")
134
135 xr = x0
136 print(f"\n Από τη λύση του χαλαρωμένου x_LP = ({','.join(str(v) for v in xr)}):")
137 frac_i = fractional(xr)[0]
138 u = F(p[frac_i], w[frac_i])
139 print(f" x{frac_i+1} = {xr[frac_i]} κλασματικό => v{frac_i+1} = 0 από (5) ΚΑΙ "
140 f"w{frac_i+1} u = p{frac_i+1} από (3) => u* = {p[frac_i]}/{w[frac_i]} = {u}")
141 v = []
142 for i in range(n):
143     vi = max(F(0), p[i] - w[i] * u)
144     v.append(vi)
145     if xr[i] == 1:
146         print(f" x{i+1} = 1 => {w[i]}*{u} + v{i+1} = {p[i]} => v{i+1} = {vi}")
147     elif xr[i] == 0:
148         print(f" x{i+1} = 0 και w{i+1}u = {w[i]*u} >= p{i+1} = {p[i]} => v{i+1} = 0")
149 W = CAP * u + sum(v)
150 print(f"\n ΔΥΪΚΗ ΛΥΣΗ: u* = {u}, v* = ({','.join(str(t) for t in v)})")
151 print(f" W* = 35*{u} + {sum(v)} = {W} = zLP = {z0} -> ισχυρή δυϊκότητα ✓")
152
153 # =====
154 # γ() ΑΚΕΡΑΙΟ ΔΥΪΚΟ ΚΑΙ ΕΛΕΓΧΟΣ CS ΓΙΑ ΤΟ ΑΚΕΡΑΙΟ ΖΕΥΓΟΣ
155 # =====
156 print("\n" + "=" * 94)
157 print("γ() ΑΚΕΡΑΙΟ ΔΥΪΚΟ ΚΑΙ ΕΛΕΓΧΟΣ CS")
158 print("=" * 94)
159 d = pulp.LpProblem("int_dual", pulp.LpMinimize)
160 U = pulp.LpVariable("u", lowBound=0, cat="Integer")

```

```

161 V = [pulp.LpVariable(f"v{i+1}", lowBound=0, cat="Integer") for i in range(n)]
162 d += CAP * U + pulp.lpSum(V)
163 for i in range(n):
164     d += w[i] * U + V[i] >= p[i]
165 d.solve(pulp.PULP_CBC_CMD(msg=0))
166 ui = int(U.value())
167 vi = [int(t.value()) for t in V]
168 Wi = CAP * ui + sum(vi)
169 print(f" u* = {ui}, v* = {vi}, W*_int = {Wi}")
170
171 print(f"\n {'i':>3}{'w_i u':>8}{'v_ιάθροισμα':>6}{'':>10}{'p_ιπερίσσεια':>6}{'':>11}{'x_i*':>6}")
172 for i in range(n):
173     tot = w[i] * ui + vi[i]
174     print(f" {'i+1':>3}{w[i]*ui:>8}{vi[i]:>6}{tot:>10}{p[i]:>6}{tot-p[i]:>11}{int(best_x[i]):>6}")
175
176 load = sum(w[i] for i in range(n) if best_x[i] == 1)
177 print(f"\n Έλεγχος (3): x_i = 1 => w_i u + v_i = p_i : "
178       f"{all(w[i]*ui+vi[i] == p[i] for i in range(n) if best_x[i] == 1)} ✓")
179 print(f" Έλεγχος (2): v_i > 0 => x_i = 1 : "
180       f"{all(best_x[i] == 1 for i in range(n) if vi[i] > 0)} ✓")
181 print(f" Έλεγχος (1): u = {ui} > 0 => Σ w_i x_i = 35 ; στην πραγματικότητα "
182       f"Σ w_i x_i = {load} < 35 -> Η CS ΠΑΡΑΒΙΑΖΕΤΑΙ")
183 print(f"\n Ερμηνεία: οι σχέσεις CS είναι ισοδύναμες με τη βελτιστότητα ΜΟΝΟ για το")
184 print(f" ΣΥΝΕΧΕΣ ζεύγος πρωτεύοντος δυϊκού-. Εδώ z*_int = {incumbent} < {Wi} = W*_int,")
185 print(f" δηλ. υπάρχει κενό ακεραιότητας (integrality gap) = {Wi - incumbent},")
186 print(f" οπότε δεν μπορεί να ισχύουν όλες οι σχέσεις CS ταυτόχρονα.")
187
188 # επαλήθευση της ακέραιας πρωτεύουσας λύσης με solver
189 q = pulp.LpProblem("knap", pulp.LpMaximize)
190 X = [pulp.LpVariable(f"x{i+1}", cat="Binary") for i in range(n)]
191 q += pulp.lpSum(p[i] * X[i] for i in range(n))
192 q += pulp.lpSum(w[i] * X[i] for i in range(n)) <= CAP
193 q.solve(pulp.PULP_CBC_CMD(msg=0))
194 print(f"\n επαλήθευση[] ακέραιο πρωτεύον με CBC: "
195       f"x = ({', '.join(str(int(t.value())) for t in X)}), z = {pulp.value(q.objective)}")

```